Python for Finance and Optimisation

Mohamed Mrad

M2-IRFA & **MMMEF** 2026-2027
Sorbonne Paris 1

Table des matières

General Introduction 5

1 Foundations of Quantitative Finance & Data Engineering 7
Chapter Introduction 7
1.1 Microstructural Concepts : Nature of Market Data 8
1.1.1 Bid, Ask, Mid-Price and Bid-Ask Spread 8
1.1.2 Execution Price and VWAP 8
1.2 Mathematical Formalization of Returns 8
1.2.1 Non-Additivity of Arithmetic Returns vs. Additivity of Logarithmic Returns 8
1.2.2 The Asset Aggregation Trap 9
1.3 Data Engineering : Ingestion, Synchronization and Cleaning of Noisy Data 9
1.3.1 Geographical Desynchronization and the Epps Effect 9
1.3.2 Anomaly Cleaning : Rolling Z-Score Filter 9
1.3.3 Python Toolkit for Data Ingestion 10
1.4 Modeling Time-Varying Volatility : EWMA 14
1.4.1 Python Toolkit for EWMA 14
1.5 Matrix Algebra and Co-Movement Spaces 17
1.6 Case Study : Temporal Sampling Bias (Graded Assignment) 17
1.6.1 Validation Algorithm 18
1.6.2 Reflection Questions for Students 18

2 Risk Modeling, Non-Gaussian Estimation 19
Chapter Introduction 19
2.1 The Limitations of Gaussian Finance 20
2.1.1 The Stylized Facts of Mandelbrot and Fama 20
2.1.2 Mathematical Formalization of Higher-Order Moments 20
2.2 Statistical Analysis of the Distribution in Python 20
2.2.1 Python Toolbox : The `scipy.stats` Submodule 20
2.3 The Jarque-Bera normality test : 25
2.4 The Shapiro–Wilk Normality Test 26
2.5 Regulatory Risk Measures : VaR and CVaR 31
2.5.1 Mathematical Interpretation of Cornish-Fisher CVaR 35

3 Modern Portfolio Theory & Optimization 37
3.0.1 Introduction to the Markowitz Framework 37
3.0.2 Formulation 1 : Risk Minimization under a Target Return Constraint . . . 38
3.0.3 Formulation 2 : Sharpe Ratio Maximization 39
3.0.4 Formulation 3 : The Sharpe Ratio along the Efficient Frontier 41
3.0.5 Graphical Summary 42
3.0.6 SLSQP Method — Sequential Least Squares Programming 43

3.0.7 Some Remarks . . . 44
3.0.8 Complete Scripted Implementation . . . 45
3.1 Machine-Based Practical Work (Exercises) . . . 52
3.1.1 Solution . . . 52

4 Asset Pricing Models (CAPM) and Factor Analysis 57
4.1 Introduction . . . 57
4.2 The Mathematical Framework . . . 58
4.2.1 The CAPM (Capital Asset Pricing Model) . . . 58
4.2.2 Multi-Factor Models : The Fama-French Three-Factor Model . . . 59
4.2.3 Non-Parametric Approach : Principal Component Analysis (PCA) . . . 60
4.2.4 Some Remarks . . . 61
4.3 Python Implementation . . . 62
4.3.1 Key Functions . . . 62
4.3.2 Python Code . . . 66
4.4 Exercises . . . 73

General Introduction

Context and the Quantitative Revolution

Since the pioneering work of Harry Markowitz in 1952 on portfolio selection, the financial industry has undergone an unprecedented technological and mathematical transformation. Modern finance is no longer merely a discipline based on accounting valuation or macroeconomic intuition ; it has become quantitative, algorithmic, and data-driven.

Today, asset managers, investment banks, and *hedge funds* operate in an ecosystem where information spreads at the speed of light. Faced with the proliferation of asset classes (equities, bonds, derivatives, commodities, crypto-assets), the ability to model risk, capture market comovements, and allocate capital optimally has become a critical competitive advantage.

Why Python for Quantitative Finance ?

For several decades, the financial computing landscape was divided :

- **C++** dominated the *Front Office* for high-performance computing (exotic option pricing, high-frequency trading) due to its execution speed.
- **MATLAB and R** were favored by researchers (*Quants*) for statistical prototyping and econometrics.

Python has established itself as the industry's de facto standard by enabling a unique convergence. Although it is an interpreted language (theoretically slower than C++), Python relies on low-level libraries written in C/C++ (`NumPy`, `SciPy`, `Pandas`). This allows financial engineers to benefit from an expressive, readable, and extremely fast-to-develop syntax, while maintaining matrix computation performance close to native code.

Furthermore, Python's native integration with the *Machine Learning* ecosystem (`Scikit-Learn`, `PyTorch`) and its ability to handle large-scale data make it an essential tool for modern financial engineering.

Course Philosophy and Learning Objectives

The objective of this 18-hour course is to bridge the gap between the rigor of financial mathematical models and the reality of their computational implementation. Theoretical optimization often suffers from an "ivory tower" syndrome : textbook models assume perfect markets, Gaussian returns, and infinite liquidity.

This course is built around a pragmatic philosophy : **confront theory with the frictions of the real world**. The curriculum is structured around three fundamental pillars :

1. ***Data Engineering* and Market Infrastructure (Part I) :** Learn how to collect, clean, and synchronize heterogeneous financial data. We will examine how to handle market anomalies (*bad ticks*), deal with asynchronous market closures across global exchanges, and model the time-varying nature of volatility (EWMA).
2. **Advanced Risk Modeling (Part II) :** Move beyond the restrictive assumption of the Normal distribution. Real financial markets are characterized by frequent crashes and asymmetric behavior. We will learn how to model these "fat tails" and implement regulatory-grade risk measures (Basel III / Solvency II), such as *Value-at-Risk* (VaR) and *Expected Shortfall* (CVaR).
3. **Constrained Optimization Algorithms (Part III) :** Translate asset allocation strategies into numerical optimization problems. Beyond the classical Markowitz model, we

will code portfolios that are robust to market noise, including the Black-Litterman model, Equal Risk Contribution (*ERC*) portfolios, and approaches based on unsupervised *Machine Learning*.

At the end of this course, students will be able to design, code from end to end, and *backtest* (test against historical data) an automated and robust investment strategy suitable for a fund's production environment.

Prerequisites

To get the most out of this material, students are expected to have :

— A solid understanding of the fundamentals of Python programming (data structures, loops, functions).

— Basic knowledge of matrix algebra (vector products, matrix inversion) and probability/statistics (expectation, variance, normal distribution).

Chapitre 1

Foundations of Quantitative Finance & Data Engineering

Chapter Introduction

Professional Context and Challenges

Data infrastructure is the nervous system of quantitative finance. Before applying complex optimization models or Machine Learning algorithms, the Quantitative Developer or Risk Analyst must face a fundamental reality : market data are asynchronous, incomplete, noisy, and subject to the microstructural rules of the order book.
Poor data acquisition management or a misunderstanding of the nature of returns systematically invalidates the results of downstream models, according to the fundamental computing principle : *"Garbage in, Garbage out"* (corrupted input data lead to abnormal output results).
This chapter establishes the technical foundations of the course. We will cover the mathematical modeling of financial time, time zone management, and the essential Data Engineering tools required to build a clean, robust, and production-ready data ingestion pipeline.

Learning Objectives

By the end of this chapter, students will be able to :

1. Master the mathematical and economic distinction between arithmetic and logarithmic returns.
2. Identify and correct geographical desynchronization biases in global markets.
3. Design a complete Python pipeline to filter price anomalies (*bad ticks*) and align heterogeneous time series.
4. Model the unstable dynamics of short-term volatility using a recursive EWMA-type estimator.

Keywords

Mid-Price, Epps Effect, Log-Returns, Forward Fill, EWMA, Frobenius Norm, Rolling Z-Score.

1.1 Microstructural Concepts : Nature of Market Data

Before executing optimization algorithms, the quantitative developer must understand the infrastructure of data originating from the order book. A single price does not exist in professional markets.

1.1.1 Bid, Ask, Mid-Price and Bid-Ask Spread

At any time t, the market presents two price streams :
— P_t^{bid} (Bid Price) : The highest price at which a buyer is willing to purchase the asset.
— P_t^{ask} (Ask Price) : The lowest price at which a seller is willing to sell the asset.
By definition, $P_t^{\text{ask}} > P_t^{\text{bid}}$. The difference between these two prices is called the **Bid-Ask Spread** :

$$S_t = P_t^{\text{ask}} - P_t^{\text{bid}} \tag{1.1}$$

The spread is a direct indicator of market illiquidity. For modeling purposes, the **Mid-Price** is frequently used :

$$P_t^{\text{mid}} = \frac{P_t^{\text{ask}} + P_t^{\text{bid}}}{2} \tag{1.2}$$

1.1.2 Execution Price and VWAP

Daily databases (such as Yahoo Finance) provide the transaction closing price (*Close*). For large order volumes, portfolio managers use the **VWAP** (*Volume Weighted Average Price*), which weights each execution price by the traded volume over the period :

$$P_{\text{VWAP}} = \frac{\sum_{i=1}^{M} P_i \times V_i}{\sum_{i=1}^{M} V_i} \tag{1.3}$$

1.2 Mathematical Formalization of Returns

The evolution of an asset price P_t is modeled by a non-stationary stochastic process. Moving to returns makes it possible to work with stationary series (statistical properties that remain invariant over time).

1.2.1 Non-Additivity of Arithmetic Returns vs. Additivity of Logarithmic Returns

The arithmetic return over the period $[0, T]$, composed of T sub-periods, is :

$$R_{0\to T} = \frac{P_T - P_0}{P_0} = \prod_{t=1}^{T}(1 + R_t) - 1 \tag{1.4}$$

The logarithmic return (or log-return) is defined as :

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1}) \tag{1.5}$$

Time additivity can be demonstrated by telescoping the sum :

$$r_{0\to T} = \ln\left(\frac{P_T}{P_0}\right) = \ln\left(\frac{P_1}{P_0} \times \frac{P_2}{P_1} \times \cdots \times \frac{P_T}{P_{T-1}}\right) = \sum_{t=1}^{T} r_t \tag{1.6}$$

1.2.2 The Asset Aggregation Trap

The arithmetic return of a portfolio (R_p) is strictly the linear combination of the arithmetic returns of its N components, weighted by the weights w_i :

$$R_p = \sum_{i=1}^{N} w_i R_i \tag{1.7}$$

Conversely, due to the non-linearity of the logarithmic function ($\ln(A+B) \neq \ln(A)+\ln(B)$), the logarithmic return of a portfolio is **not** equal to the weighted sum of log-returns :

$$r_p \neq \sum_{i=1}^{N} w_i r_i \tag{1.8}$$

1.3 Data Engineering : Ingestion, Synchronization and Cleaning of Noisy Data

1.3.1 Geographical Desynchronization and the Epps Effect

When optimizing assets traded across different time zones (e.g., Paris and New York), trading windows overlap only partially. If a raw daily correlation matrix is calculated without adjustment, the coefficients are subject to an artificial downward bias known as the **Epps Effect**.

1.3.2 Anomaly Cleaning : Rolling Z-Score Filter

Real-world data feeds contain *bad ticks* (data-entry errors or abnormal price jumps caused by API bugs (Software interface bugs)). We implement a filter based on a rolling Z-Score to remove returns that deviate by more than 3 standard deviations from the local mean.

Z-Score

The **Z-Score** (or *standard score*) is a fundamental statistical tool. It measures **how many standard deviations an observation lies away from the mean of a group**.
It is an essential standardization method for comparing variables that come from completely different distributions or scales.

Practical Use and Concrete Example Imagine that you take two different exams :
- In **Mathematics**, you obtain **15/20** (the class average is 13).
- In **French**, you obtain **13/20** (the class average is 10).

To determine in which subject you actually performed better relative to the rest of the students, simply comparing the deviation from the mean is not sufficient. The dispersion of the grades (the standard deviation) must also be taken into account. This is precisely what the Z-Score allows us to do.

Mathematical Formula To calculate the Z-Score of an observation, the following formula is used :

$$Z = \frac{x - \mu}{\sigma} \tag{1.9}$$

Where :
- x is the value of the observation being studied (e.g., your grade).
- μ represents the **mean** of the population or group.
- σ represents the **standard deviation**, which measures the dispersion of the data around the mean.

Interpretation of Results The result is a real number (positive, negative, or zero) and can be interpreted as follows :
- $Z = 0$: The value is exactly equal to the group mean.
- $Z > 0$ (e.g., $+1.5$) : The value is above the mean. A score of $+1.5$ means that the observation lies 1.5 standard deviations above the mean.
- $Z < 0$ (e.g., -2) : The value is below the mean.

The Empirical Rule (Normal Distribution) For a normal distribution (Gaussian curve) :
- Approximately **68%** of observations have a Z-Score between -1 and $+1$.
- Approximately **95%** of observations have a Z-Score between -2 and $+2$.
- A Z-Score greater than $+3$ or lower than -3 is extremely rare (less than 1% probability), making it very useful for detecting anomalies or outliers (*outliers*).

1.3.3 Python Toolkit for Data Ingestion

- `yf.download()` : Queries and downloads financial data.

Example

The following line of code makes it possible to download historical data for the different financial assets under study :

```
raw_prices = yf.download(tickers, start="2022-01-01", end="2026-01-01")["Adj Close"]
```

The function `yf.download()` belongs to the `yfinance` library. It retrieves historical market data available from Yahoo Finance.
The `tickers` variable contains the different assets being studied :

$$\texttt{tickers} = [\texttt{'AAPL'}, \texttt{'\^{}FCHI'}, \texttt{'\^{}N225'}, \texttt{'BTC-USD'}]$$

These codes correspond respectively to Apple, the CAC 40, the Nikkei 225, and Bitcoin expressed in U.S. dollars.
The argument

```
start="2022-01-01"
```

indicates the start date of the period under study, while

```
end="2026-01-01"
```

indicates the end date. Since the date specified in `end` is generally exclusive, the period considered is therefore :

$$\boxed{\text{2022-01-01} \leq t < \text{2026-01-01}}$$

The function `yf.download()` retrieves several pieces of information for each asset, including :
— `Open` : opening price ;
— `High` : highest price of the trading session ;
— `Low` : lowest price of the trading session ;
— `Close` : closing price ;
— `Adj Close` : adjusted closing price ;
— `Volume` : traded volume.
The expression

```
["Adj Close"]
```

then selects only the adjusted closing prices. The other information is therefore removed from the DataFrame.
The resulting structure is a dataset in which rows correspond to dates and columns correspond to the different assets :

Date	AAPL	CAC 40	Nikkei 225	BTC-USD
t_1	$P_{1,\text{AAPL}}$	$P_{1,\text{CAC}}$	$P_{1,\text{Nikkei}}$	$P_{1,\text{BTC}}$
t_2	$P_{2,\text{AAPL}}$	$P_{2,\text{CAC}}$	$P_{2,\text{Nikkei}}$	$P_{2,\text{BTC}}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

The variable `raw_prices` therefore contains a `pandas DataFrame` grouping the adjusted closing prices of the different assets over the period under study.
The use of adjusted prices is particularly appropriate for return analysis, as they account for certain events affecting price series, notably dividends and stock splits (*splits*).
These data can then be used to calculate logarithmic returns :

$$\boxed{r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)}$$

where P_t denotes the adjusted price at date t.
— `df.rolling().mean()` and `df.rolling().std()` : Compute the local statistics required for the rolling Z-Score.
— `np.abs()` : Returns the absolute value to identify two-sided deviations.
— **Filling missing values with `ffill()` and `bfill()`**
The following expression, used with the `pandas` library,

```
raw_prices.ffill().bfill()
```

allows missing values (NaN, meaning "Not a Number") in a data series to be filled. It combines two successive operations :

1. `ffill()` (*forward fill*) ;
2. `bfill()` (*backward fill*).

1. `ffill()` : Forward filling. The `ffill()` method replaces each missing value with

the last known value located before it.
For example, consider the following series :

$$[100,\ \text{NaN},\ \text{NaN},\ 105,\ \text{NaN},\ 110].$$

After applying `ffill()`, we obtain :

$$[100,\ 100,\ 100,\ 105,\ 105,\ 110].$$

Thus, for a position i containing a missing value, `ffill()` uses the last available observation before i.

2. `bfill()` : Backward filling.
The `bfill()` method replaces a missing value with the first known value located after it.
For example :

$$[\text{NaN},\ \text{NaN},\ 100,\ 105,\ \text{NaN}]$$

becomes, after applying `ffill()` :

$$[\text{NaN},\ \text{NaN},\ 100,\ 105,\ 105].$$

The first two values remain missing because no observation is available before them. Applying `bfill()` then makes it possible to complete them :

$$[100,\ 100,\ 100,\ 105,\ 105].$$

3. Combining the two methods
The expression

$$\boxed{\texttt{raw_prices.ffill().bfill()}}$$

therefore performs the following operations :

$$\texttt{raw_prices} \xrightarrow{\texttt{ffill()}} \text{forward filling} \xrightarrow{\texttt{bfill()}} \text{filling of remaining missing values.}$$

Its behavior can be summarized as follows :

$$\text{NaN} \longrightarrow \begin{cases} \text{last known value before,} & \text{if it exists,} \\ \text{first known value after,} & \text{otherwise.} \end{cases}$$

This method is particularly useful for time series, for example when some price observations are missing. It therefore makes it possible to obtain a series containing no missing values, provided that at least one observation is known in the series.

```python
import yfinance as yf # Yahoo Finance
import pandas as pd # manipulate, clean, analyze and organize data
import numpy as np # mainly used for numerical computation

# Ingestion of an international and multi-asset portfolio
tickers = ['AAPL', '^FCHI', '^N225', 'BTC-USD']
raw_prices = yf.download(tickers, start="2022-01-01", end="2026-01-01")['Adj Close']

# AAPL (Apple stock), '^FCHI' (CAC 40 index),
# '^N225' (Japanese Nikkei 225 index),
```

```python
# BTC-USD (Bitcoin in US dollars)

# Filling liquidity gaps (specific public holidays)
prices_cleaned = raw_prices.ffill().bfill()

# Calculation of log returns
log_rets = np.log(prices_cleaned / prices_cleaned.shift(1)).dropna()

# BAD TICK FILTER: 20-day rolling Z-Score
rolling_mean = log_rets.rolling(window=20).mean()
rolling_std = log_rets.rolling(window=20).std()
z_scores = np.abs((log_rets - rolling_mean) / rolling_std)

# Replace extreme returns (Z-Score > 3) with 0 (price stagnation)
log_rets_filtered = log_rets.mask(z_scores > 3, 0)

print(f"Number of anomalies filtered and corrected: {(z_scores > 3).sum().sum()}
    ")
```

Listing 1.1 – Market Data Ingestion and Anomaly Filtering Pipeline

Note : Counting Outliers with `sum()`
The following Python expression is used to count the total number of values whose Z-Score is greater than 3 :

`(z_scores > 3).sum().sum()`

The presence of two calls to `sum()` is explained by the fact that `z_scores` is generally a `pandas DataFrame`, i.e., a two-dimensional data structure composed of rows and columns.
The condition

`z_scores > 3`

compares each value in the DataFrame with 3. It produces a matrix of Boolean values :

$$\begin{pmatrix} 2.1 & 4.5 & 1.2 \\ 3.8 & 1.5 & 5.2 \end{pmatrix} \longrightarrow \begin{pmatrix} \texttt{False} & \texttt{True} & \texttt{False} \\ \texttt{True} & \texttt{False} & \texttt{True} \end{pmatrix}.$$

In Python, Boolean values are interpreted as numbers when performing a sum :

$$\texttt{True} = 1, \qquad \texttt{False} = 0.$$

The previous matrix can therefore be represented as :

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}.$$

First `sum()`

The first call to `sum()` performs a column-wise sum :

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \xrightarrow{\texttt{sum()}} \begin{pmatrix} 1 & 1 & 1 \end{pmatrix}.$$

This gives the number of values greater than 3 in each column.

Second `sum()`

The second call to `sum()` then adds the results obtained for each column :

$$1 + 1 + 1 = 3.$$

We therefore obtain the total number of values greater than 3 in the entire DataFrame.
Thus,

$$\boxed{\texttt{(z_scores > 3).sum().sum()} = \text{total number of values such that } z > 3}$$

The first `sum()` performs the sum across columns, while the second adds the results obtained for all columns.

Note. If `z_scores` is a `Series` rather than a `DataFrame`, a single call to `sum()` is generally sufficient :

$$\boxed{\texttt{(z_scores > 3).sum()}}$$

Note. `dropna()` is a pandas function used to remove rows or columns containing missing values (NaN).

1.4 Modeling Time-Varying Volatility : EWMA

Return volatility is not constant ; it exhibits clusters of high and low instability (the *Volatility Clustering* phenomenon). To capture this dynamic, we formalize the **EWMA** model (*Exponentially Weighted Moving Average*), which is notably used by *RiskMetrics*.
The variance at time t, denoted by σ_t^2, is an exponentially weighted average of the past variance and the most recent squared return :

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2 \tag{1.10}$$

where λ is the persistence parameter (generally set to 0.94 for daily data).

1.4.1 Python Toolkit for EWMA

— `df.ewm(alpha, adjust=False)` : Directly implements the EWMA recurrence. The parameter `alpha` is related to λ through the relationship $\alpha = 1-\lambda$. The argument `adjust=False` ensures the strict application of the recursive formula without normalization of the initial weights.

```
import matplotlib.pyplot as plt

# RiskMetrics lambda parameter
lambda_param = 0.94
alpha_param = 1 - lambda_param

# Daily EWMA variance calculation
ewma_variance = log_rets_filtered['AAPL'].pow(2).ewm(
    alpha=alpha_param,
    adjust=False
).mean()

# Annualization of EWMA volatility
```

```
ewma_vol_annualized = np.sqrt(ewma_variance * 252)

plt.figure(figsize=(12, 5))
plt.plot(
    ewma_vol_annualized,
    color='darkblue',
    label='EWMA Volatility (RiskMetrics $\lambda=0.94$)'
)
plt.title("Time-Varying Volatility Dynamics of Apple")
plt.xlabel("Date")
plt.ylabel("Annualized Volatility")
plt.legend()
plt.grid(True)
plt.show()
```

Listing 1.2 – EWMA Volatility Calculation and Visualization

Explanations : Variance Estimation Using the EWMA Method

The following line of code estimates the variance of the logarithmic returns of Apple stock (`AAPL`) using the EWMA (*Exponentially Weighted Moving Average*) method :

```
ewma_variance = log_rets_filtered['AAPL'].pow(2).ewm(
    alpha=alpha_param,
    adjust=False
).mean()
```

This expression can be broken down into several steps.

Selecting AAPL Returns

The instruction `log_rets_filtered['AAPL']` selects the time series of logarithmic returns of Apple stock.
The logarithmic return at date t is defined as :

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right),$$

where P_t represents the asset price at date t.

Squaring the Returns

The instruction `.pow(2)` squares each return :

$$r_t \longrightarrow r_t^2.$$

For example :

$$[-0.02,\ 0.01,\ -0.03] \longrightarrow [0.0004,\ 0.0001,\ 0.0009].$$

Squaring returns makes it possible, in particular, to measure their dispersion around zero and constitutes the basis for variance estimation.

Exponentially Weighted Moving Average

The instruction

```
.ewm(alpha=alpha_param, adjust=False)
```

sets up an exponentially weighted moving average (*Exponentially Weighted Moving Average*, EWMA).
This method assigns greater weight to recent observations than to older observations.
The EWMA variance can be represented by the recursive relationship :

$$\boxed{\widehat{\sigma}_t^2 = \alpha r_{t-1}^2 + (1-\alpha)\widehat{\sigma}_{t-1}^2}$$

where :

- r_{t-1} represents the logarithmic return in the previous period ;
- $\widehat{\sigma}_{t-1}^2$ represents the estimated variance in the previous period ;
- α is the smoothing parameter ;
- $0 < \alpha < 1$.

When α is high, recent observations have a greater influence on the variance estimate. Conversely, when α is low, older observations retain greater influence.

Calculation of the Mean

The instruction `.mean()` calculates the exponentially weighted mean of the squared returns. The EWMA variance can therefore be written as :

$$\boxed{\widehat{\sigma}_t^2 = \alpha r_{t-1}^2 + \alpha(1-\alpha) r_{t-2}^2 + \alpha(1-\alpha)^2 r_{t-3}^2 + \cdots}$$

The most recent observations therefore have the greatest weights.
Thus, the entire instruction

```
log_rets_filtered['AAPL'].pow(2).ewm(
    alpha=alpha_param,
    adjust=False
).mean()
```

can be summarized as :

logarithmic returns → squared returns → exponentially weighted average → EWMA variance

The variable `ewma_variance` therefore contains a time series representing the estimated conditional variance of AAPL returns.
Finally, to obtain EWMA volatility, it is sufficient to take the square root of the variance :

```
ewma_volatility = np.sqrt(ewma_variance)
```

which gives :

$$\boxed{\widehat{\sigma}_t = \sqrt{\widehat{\sigma}_t^2}}$$

EWMA volatility therefore provides a measure of volatility that assigns greater importance to recent market movements.

1.5 Matrix Algebra and Co-Movement Spaces

Let $\mathbf{r}_t = [r_{1,t}, r_{2,t}, \ldots, r_{N,t}]^T$ be the vector of returns at time t. The annualized variance-covariance matrix $\mathbf{\Sigma}$ (of dimension $N \times N$) is expressed in matrix form as :

$$\mathbf{\Sigma} = 252 \times \frac{1}{T-1} \sum_{t=1}^{T} (\mathbf{r}_t - \mu)(\mathbf{r}_t - \mu)^T \tag{1.11}$$

where μ is the vector of the average returns of each asset.
The Pearson correlation matrix is then given by :

$$\boxed{\mathbf{R} = \mathbf{D}^{-\frac{1}{2}} \mathbf{\Sigma} \mathbf{D}^{-\frac{1}{2}}}$$

where $\mathbf{\Sigma}$ denotes the covariance matrix and $\mathbf{D}$ the diagonal matrix of variances :

$$\mathbf{D} = \operatorname{diag}\left(\sigma_1^2, \sigma_2^2, \ldots, \sigma_n^2\right).$$

Thus, each element of the correlation matrix $\mathbf{R}$ is given by :

$$\boxed{\rho_{ij} = \frac{\sigma_{ij}}{\sigma_i \sigma_j}}$$

where $\sigma_{ij} = \operatorname{Cov}(r_i, r_j)$.

```python
import seaborn as sns # statistical data visualization

# Calculation of the annualized covariance matrix
Sigma = log_rets_filtered.cov() * 252

# Calculation of the Pearson correlation matrix
R = log_rets_filtered.corr()

# Professional visualization using Seaborn
plt.figure(figsize=(8, 6))
sns.heatmap(
    R,
    annot=True,
    cmap='bwr',
    vmin=-1,
    vmax=1,
    linewidths=0.5
)
plt.title("Co-Movement Matrix (Correlation $R$)")
plt.show()
```

Listing 1.3 – Matrix Calculation of Covariance and Correlation

1.6 Case Study : Temporal Sampling Bias (Graded Assignment)

Problem : An asset management model rebalances its portfolios every month. Is it preferable to estimate the risk parameters ($\mathbf{\Sigma}$) using daily or weekly returns ?

1.6.1 Validation Algorithm

The student must run the following script to measure the Frobenius norm of the difference matrix. The Frobenius norm measures the overall distance between two matrices :

$$\|\mathbf{\Sigma}_{\text{Daily}} - \mathbf{\Sigma}_{\text{Weekly}}\|_F = \sqrt{\sum_{i=1}^{N}\sum_{j=1}^{N}(\sigma_{ij,D} - \sigma_{ij,W})^2} \tag{1.12}$$

```
# 1. Daily Estimation
Sigma_daily = log_rets_filtered.cov() * 252

# 2. Weekly Estimation using specific resampling
prices_weekly = prices_cleaned.resample('W').last()
log_rets_weekly = np.log(
    prices_weekly / prices_weekly.shift(1)
).dropna()

Sigma_weekly = log_rets_weekly.cov() * 52

# 3. Calculation of the Frobenius matrix distance
matrix_diff = Sigma_daily - Sigma_weekly
frobenius_norm = np.linalg.norm(matrix_diff, ord='fro')

print("--- Quantitative Audit Results ---")
print("Difference matrix :\n", matrix_diff)
print(f"\nGlobal Frobenius distance : {frobenius_norm:.6f}")
```

Listing 1.4 – Case Study Resolution Script

1.6.2 Reflection Questions for Students

1. Why is the Frobenius distance not strictly equal to 0 even though both methods measure risk over the same historical period ?
2. What is the impact of the Epps Effect on the weekly estimation compared with the daily estimation ?

Chapitre 2

Risk Modeling, Non-Gaussian Estimation

Chapter Introduction

Professional Context and Challenges

On October 19, 1987, during « Black Monday », the Dow Jones index collapsed by 22.6% in a single trading session. Under a classical normal distribution framework, such a movement represents an event more than 20 standard deviations (20σ) away from the mean, with a probability of occurrence so small that it should not happen even on the timescale of the age of the universe. Yet the crashes of 1998 (LTCM), 2008 (Lehman Brothers), 2020 (Covid-19), or liquidity crises in crypto-assets remind us of an unavoidable mathematical reality : **real financial markets are not Gaussian**.

For a portfolio manager or banking regulator, blindly using the normal distribution is a deadly danger. It systematically leads to an underestimation of extreme loss risks. The objective of this chapter is to learn how to measure risk where it actually hides : in asymmetries and fat tails. We will formalize and implement the key indicators of institutional risk management, required by the international Basel III and Solvency II frameworks.

Learning Objectives

At the end of this chapter, students will be able to :

1. Mathematically diagnose and reject the normality assumption for an asset using the Jarque-Bera statistical test.
2. Model and interpret higher-order statistical moments (Skewness and Kurtosis) for highly volatile assets.
3. Implement in Python the algorithms for calculating *Value-at-Risk* (VaR) and *Expected Shortfall* (CVaR) using historical and parametric approaches.
4. Adjust risk calculations using the Cornish-Fisher expansion to incorporate non-Gaussian effects into capital requirements.

Keywords

Leptokurtic Distribution, Higher-Order Moments, Loss Quantile, Value-at-Risk, Expected Shortfall, Cornish-Fisher, Jarque-Bera Test.

2.1 The Limitations of Gaussian Finance

Classical financial theory (Bachelier, Markowitz, Black-Scholes) relies on the fundamental assumption that the logarithmic returns of assets follow a normal distribution (Gaussian distribution). Although this assumption greatly simplifies analytical calculations, it is systematically contradicted by real market data.

2.1.1 The Stylized Facts of Mandelbrot and Fama

As early as the 1960s, Benoît Mandelbrot and Eugene Fama highlighted the **stylized facts** of financial time series :

— **Fat Tails :** Extreme events (crashes, euphoric rallies) have a much higher probability of occurrence than predicted by the normal distribution.

— **Asymmetry (Negative Skewness) :** The return distributions of risky assets (such as stocks) often exhibit left skewness, indicating sharper downward movements than upward movements.

— **Volatility Clustering** : Volatility is not constant and propagates in clusters.

2.1.2 Mathematical Formalization of Higher-Order Moments

To quantify the deviation from normality, we analyze the centered moments of order 3 (**Skewness**) and order 4 (**Kurtosis**). Let r denote the random variable representing returns, $\mu = E[r]$ the expected value, and $\sigma = \sqrt{E[(r-\mu)^2]}$ the standard deviation.
Fisher's skewness coefficient is defined as :

$$S = E\left[\left(\frac{r-\mu}{\sigma}\right)^3\right] \tag{2.1}$$

A perfect Gaussian distribution has $S = 0$.
The kurtosis coefficient is defined as :

$$K = E\left[\left(\frac{r-\mu}{\sigma}\right)^4\right] \tag{2.2}$$

A Gaussian distribution has $K = 3$. In quantitative finance, we use the **Excess Kurtosis** ($K_{\text{ex}} = K - 3$). If $K_{\text{ex}} > 0$, the distribution is said to be **leptokurtic** : it has fat tails and a higher central peak.

2.2 Statistical Analysis of the Distribution in Python

2.2.1 Python Toolbox : The `scipy.stats` Submodule

To audit the statistical shape of our time series, we use specialized functions :

— `stats.skew(array)` : Computes the empirical skewness coefficient. A negative value indicates a left-stretched tail (more extreme latent losses).

Explanation : In quantitative finance, this statistic is particularly useful for studying the asymmetry of return distributions.
Intuitively, skewness determines whether observations are distributed symmetrically around their mean or whether one of the two tails of the distribution is more significant than the other.

The important term in the formula is the cube $(r-\mu)^3$. Unlike the square used to calculate variance, the cube preserves the sign of the deviation from the mean.
Thus, an observation located above the mean contributes positively :

$$r-\mu>0 \quad \Longrightarrow \quad (r-\mu)^3>0$$

Whereas an observation located below the mean contributes negatively :

$$r-\mu<0 \quad \Longrightarrow \quad (r-\mu)^3<0$$

Interpretation of the Sign The sign of the skewness coefficient identifies the direction of the distribution's asymmetry.
— A positive skewness corresponds to right skewness :
— A zero skewness corresponds to an approximately symmetric distribution :
— A negative skewness corresponds to left skewness :
A negative skewness therefore means that the distribution has a **longer or heavier left tail** than its right tail.

Example Consider the following return series :

$$\{-2\%,-1\%,0\%,1\%,2\%,-10\%\} \tag{2.3}$$

Most returns are relatively close to zero, but one particularly large loss of -10% is observed. This observation contributes strongly and negatively to the skewness calculation because it appears in the term :

$$(-10\%)^3<0$$

The cube amplifies the influence of observations that are very far from the mean.
Consequently, a few strongly negative returns can lead to a significantly negative skewness.

Why Is Skewness Important in Finance ? Two assets can have similar means and volatilities while exhibiting very different return distributions.
For example, one asset may exhibit relatively symmetric gains and losses, while another may exhibit many small gains but also a few extremely large losses.
In the second case, the distribution may exhibit negative skewness : $\text{Skewness}(r)<0$.
This indicates that the distribution is asymmetric toward extreme negative returns.
Skewness therefore provides information complementary to volatility. Volatility mainly measures the dispersion of returns around their mean, while skewness provides information about the **asymmetry of the distribution**.

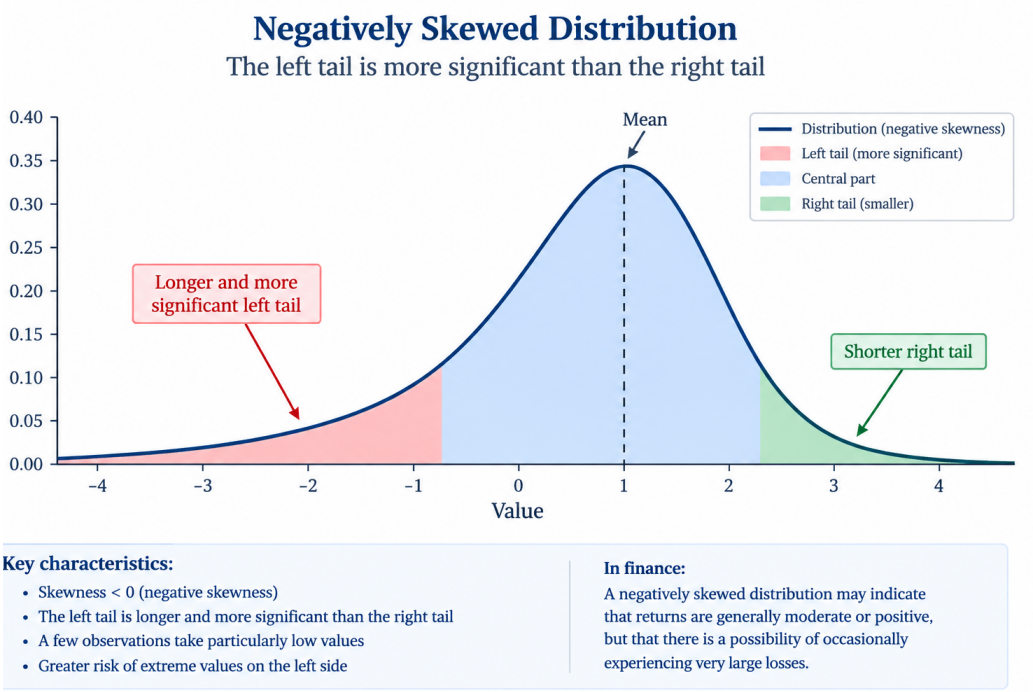


FIGURE 2.1 – Illustration of a negatively skewed distribution.

— `stats.kurtosis(array, fisher=True)` : allows the calculation of the kurtosis coefficient of a data series. In quantitative finance, kurtosis is particularly useful for studying the presence of extreme values in a return distribution. It can notably identify distributions exhibiting *fat tails*.

Example : The important term in the formula is :

$$(r - \mu)^4$$

The fourth power gives particularly strong importance to observations that are very far from the mean.
For example, consider two deviations from the mean :

$$r_1 - \mu = 1 \quad \text{and} \quad r_2 - \mu = 4.$$

With a power of two, the contributions would be :

$$1^2 = 1 \quad \text{and} \quad 4^2 = 16.$$

With a power of four, they become :

$$1^4 = 1 \quad \text{and} \quad 4^4 = 256 \tag{2.4}$$

The observation located four standard deviations from the mean therefore has a much greater influence on kurtosis. Kurtosis is thus particularly sensitive to extreme observations.

The Fisher Convention There are two conventions commonly used to define kurtosis. Pearson's kurtosis is given by :

$$K = \frac{\mathbb{E}\left[(r-\mu)^4\right]}{\sigma^4} \tag{2.5}$$

For a normal distribution, this quantity is $K = 3$. The Fisher convention consists of subtracting 3 from Pearson's kurtosis :

$$K_F = K_{ex} = \frac{\mathbb{E}\left[(r-\mu)^4\right]}{\sigma^4} - 3 \tag{2.6}$$

Thus, for a normal distribution :

$$K_F = 3 - 3 = 0 \tag{2.7}$$

This is the convention used when writing : `stats.kurtosis(array, fisher=True)`.
The parameter `fisher=True` therefore instructs the function to return the **excess kurtosis** rather than Pearson's kurtosis.

Interpretation of the Result When `fisher=True`, the reference value is therefore :

$$K_F = 0$$

The result can then be interpreted as follows :

$$K_F > 0 \quad \Longrightarrow \quad \text{fatter tails than those of a normal distribution}$$

$$K_F = 0 \quad \Longrightarrow \quad \text{kurtosis comparable to that of a normal distribution}$$

$$K_F < 0 \quad \Longrightarrow \quad \text{thinner tails than those of a normal distribution}$$

A distribution with positive Fisher kurtosis is generally described as **leptokurtic**.

What Does "Fat Tails" Mean ? A distribution has fat tails when it contains more observations very far from its mean than would be predicted by a normal distribution with the same mean and variance.
Consider, for example, a return series :

$$\{-10\%, -1\%, 0\%, 1\%, 12\%\}$$

The returns of -10% and 12% are very far from the mean.
Because of the fourth power present in the kurtosis formula, these observations have a significant influence :

$$(-10\%)^4 \quad \text{and} \quad (12\%)^4.$$

Thus, the presence of a few extremely large in absolute value returns can strongly increase kurtosis.

Financial Interpretation This property is particularly important when analyzing financial returns. In many financial markets, returns do not perfectly follow a normal distribution. They may exhibit **fat tails**, meaning that extreme events occur more frequently than predicted by a normal distribution.
For example, a model based on a normal distribution may assign a very low probability to a return of -8% whereas such an event may be less rare in actual financial data.
Positive kurtosis therefore highlights this characteristic :

$$K_F > 0 \quad \Longrightarrow \quad \text{fat tails} \quad \Longrightarrow \quad \text{more extreme observations}$$

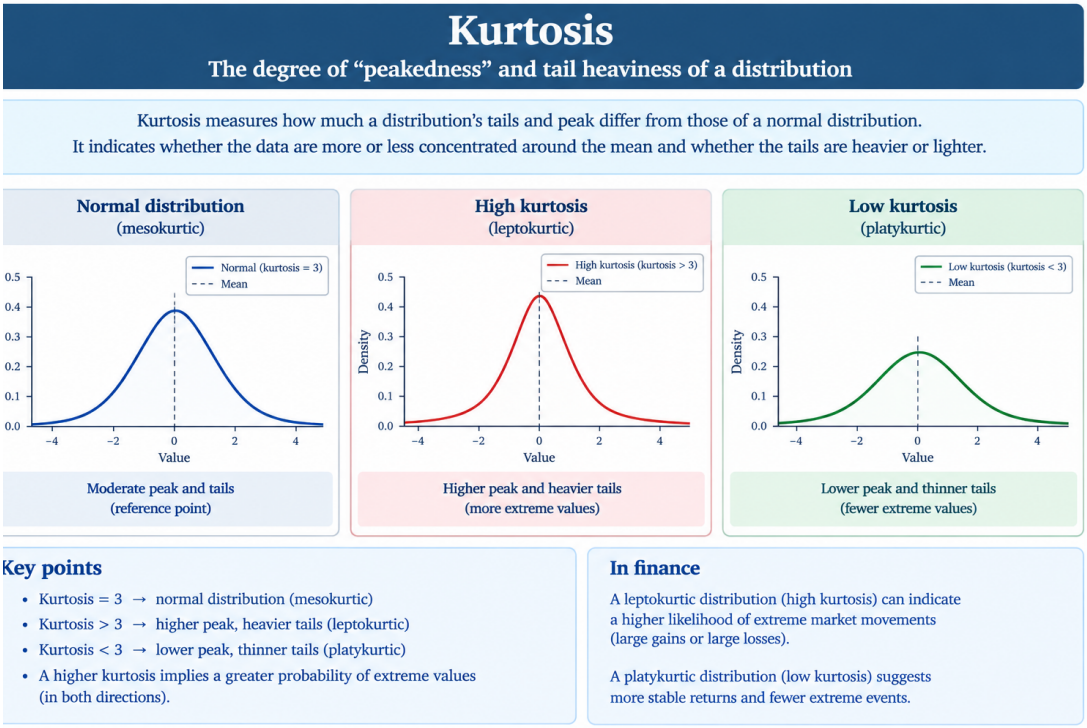


FIGURE 2.2 – Illustration of a negatively skewed distribution.

Difference Between Skewness and Kurtosis It is important to distinguish skewness from kurtosis.
Skewness primarily measures the asymmetry of the distribution :

$$\text{Skewness}(r) < 0 \quad \Longrightarrow \quad \text{asymmetry toward negative returns} \tag{2.8}$$

Kurtosis primarily measures the importance of the distribution tails and the presence of extreme observations :

$$K_F > 0 \quad \Longrightarrow \quad \text{fatter tails and more extreme values} \tag{2.9}$$

We can therefore remember :

$$\text{Skewness} \quad \longrightarrow \quad \text{distribution asymmetry} \tag{2.10}$$

and :

$$\text{Kurtosis} \quad \longrightarrow \quad \text{importance of extreme events} \tag{2.11}$$

An asset can therefore simultaneously exhibit negative skewness and positive kurtosis :

$$\text{Skewness}(r) < 0 \qquad \text{and} \qquad K_F > 0 \tag{2.12}$$

In this case, the return distribution exhibits both **asymmetry toward losses** and **fat tails**.
This combination is particularly important in risk management because it indicates that extreme losses can be both asymmetric and more frequent than suggested by a normal distribution.

2.3 The Jarque-Bera normality test :

Normality test based on the formula $JB = \frac{T}{6}\left(S^2 + \frac{K_{\text{ex}}^2}{4}\right)$, where :
— T represents the number of observations ;
— S represents skewness ;
— K represents excess kurtosis.

The statistic therefore combines the deviation from zero skewness and the deviation from zero excess kurtosis.
The function `stats.jarque_bera(.)` returns two values : the test statistic and the *p-value*. If the *p-value* < 0.05, the normality hypothesis is rejected at the 95% confidence level.

Explanation : The **Jarque-Bera** test determines whether a data series is compatible with a **normal distribution**. It is particularly useful in finance for checking whether returns can reasonably be modeled using a normal distribution.
The test is based on two characteristics of the distribution : **skewness** and **kurtosis**.
Recall that for a normal distribution : Skewness $= 0$ and and, when using Fisher's excess kurtosis : Kurtosis $= 0$, so that $JB = 0$.

Test Hypotheses The null hypothesis is :

$$H_0 : \text{the distribution of the data is normal} \tag{2.13}$$

The alternative hypothesis is :

$$H_1 : \text{the distribution of the data is not normal} \tag{2.14}$$

Interpretation of the p-value A significance level of 5% is generally used :

$$\alpha = 0.05 \tag{2.15}$$

If :

$$p\text{-value} < 0.05 \tag{2.16}$$

then H_0 is rejected and we conclude that the data exhibit characteristics significantly different from those of a normal distribution.
Conversely, if :

$$p\text{-value} > 0.05 \tag{2.17}$$

we do not reject H_0. This means that the test does not provide sufficient evidence to reject the normality hypothesis.

p-value of the Jarque–Bera test Under the null hypothesis of normality, and asymptotically,

$$\boxed{JB \overset{a}{\sim} \chi^2(2)}$$

that is, the Jarque–Bera statistic asymptotically follows a χ^2 distribution with two degrees of freedom. The p-value then corresponds to the probability, under the null hypothesis, of observing a test statistic at least as large as the one obtained from the sample :

$$P\left(\chi_2^2 \geq JB_{\text{obs}}\right)$$

or, equivalently, using the cumulative distribution function $F_{\chi_2^2}$ of the χ^2 distribution with two degrees of freedom :

$$1 - F_{\chi_2^2}(JB_{\text{obs}})$$

In Python, the test can be performed using :

```
jb_stat, p_value = stats.jarque_bera(log_rets_filtered['BTC-USD'])
```

```python
import yfinance as yf
import pandas as pd
import numpy as np
import scipy.stats as stats

# Data ingestion for a highly volatile asset (e.g., Bitcoin)
ticker = 'BTC-USD'
prices = yf.download(ticker, start="2021-01-01", end="2026-01-01")['Adj Close'].ffill()
log_returns = np.log(prices / prices.shift(1)).dropna()

# Calculation of distribution moments via scipy.stats
skewness_val = stats.skew(log_returns)
kurtosis_ex = stats.kurtosis(log_returns, fisher=True)

# Execution of the Jarque-Bera test
jb_stat, p_value = stats.jarque_bera(log_returns)

print(f"--- Statistical Analysis of {ticker} ---")
print(f"Skewness : {skewness_val:.4f} (Expected for a normal distribution: 0)")
print(f"Excess Kurtosis    : {kurtosis_ex:.4f}  (Expected for a normal distribution: 0)")
print(f"Jarque-Bera Statistic : {jb_stat:.2f}")
print(f"Test p-value       : {p_value:.6f}")

if p_value < 0.05:
    print("Conclusion: We reject H0. The distribution is non-Gaussian.")
else:
    print("Conclusion: We cannot reject H0.")
```

Listing 2.1 – Normality Test and Higher-Order Moment Analysis

2.4 The Shapiro–Wilk Normality Test

The Shapiro–Wilk test is another statistical test used to determine whether a sample is compatible with a normal distribution.
In the financial context, this test can be used to assess whether the observed returns of an asset or portfolio can reasonably be considered normally distributed.

Test hypotheses Consider a sample of returns :

$$X_1, X_2, \ldots, X_n.$$

The Shapiro–Wilk test is based on the following hypotheses :

$$\boxed{\begin{array}{l} H_0 : X \sim \mathcal{N}(\mu, \sigma^2), \\ H_1 : X \text{ does not follow a normal distribution.} \end{array}}$$

The null hypothesis H_0 therefore means that the observations are compatible with a normal distribution. The objective of the test is to determine whether the data provide sufficient evidence to reject this hypothesis.

General principle of the test The idea behind the Shapiro–Wilk test is to compare the observations in the sample with what we would expect if the observations actually came from a normal distribution.
The first step consists of ordering the observations in ascending order :

$$X_{(1)} \leq X_{(2)} \leq \cdots \leq X_{(n)}.$$

The $X_{(i)}$ are called the **order statistics**. For example, for the sample :

$$\{0.03, -0.01, 0.02, 0.05, -0.02\},$$

we obtain :

$$X_{(1)} = -0.02, \quad X_{(2)} = -0.01, \quad X_{(3)} = 0.02, \quad X_{(4)} = 0.03, \quad X_{(5)} = 0.05.$$

The test then compares this ordered structure with what would be observed in a sample drawn from a normal distribution.

The Shapiro–Wilk statistic The test statistic is given by :

$$\boxed{W = \frac{\left(\sum_{i=1}^{n} a_i X_{(i)}\right)^2}{\sum_{i=1}^{n} (X_i - \bar{X})^2}}$$

where :

- $X_{(i)}$ denotes the i-th observation after ordering ;
- $\bar{X}$ is the sample mean ;
- a_i are the coefficients of the Shapiro–Wilk test. They are chosen to efficiently measure the deviation between the observed distribution and a normal distribution.
- n is the sample size.

The statistic W generally lies between 0 and 1.

- A value of W close to 1 indicates that the structure of the data is compatible with what is expected under a normal distribution.
- Conversely, a lower value of W indicates a greater deviation from normality.
- However, the final decision of the test is generally based on the **p-value**, rather than simply on whether W is close to 1.

p-value of the Shapiro–Wilk test Once the observed statistic W_{obs} has been calculated, the p-value corresponds to the probability, under the null hypothesis, of obtaining a value of W at least as small as the one observed :

$$P\left(W \leq W_{\text{obs}} \mid H_0\right)$$

The direction of the inequality is important. In the Shapiro–Wilk test, values of W close to 1 are associated with data that are more consistent with a normal distribution, whereas low values of W indicate a deviation from normality. Unlike the Jarque–Bera test, the statistic W does not follow a simple standard distribution such as a normal distribution or a χ^2 distribution. The distribution of W under the null hypothesis depends notably on the sample size and is mathematically complex because it does not have a simple analytical form. Therefore, the p-value is obtained using numerical approximations implemented in statistical software.

Where do the coefficients a_i come from ? The coefficients a_i are the most technical part of the test. The idea is to construct coefficients that allow us to best compare the ordered observations in the sample with what we would expect to observe under a normal distribution. To understand their origin, consider a sample of independent random variables following a standard normal distribution :

$$Z_1, Z_2, \ldots, Z_n \sim \mathcal{N}(0,1).$$

These observations are also ordered :

$$Z_{(1)} \leq Z_{(2)} \leq \cdots \leq Z_{(n)}.$$

We then consider the expected values of the order statistics :

$$m_i = E[Z_{(i)}].$$

The vector of expected values is therefore :

$$m = \begin{pmatrix} m_1 \\ m_2 \\ \vdots \\ m_n \end{pmatrix}.$$

This vector represents the average positions that the ordered observations should have if the data came from a standard normal distribution. For example : $m_1 < 0,\ m_n > 0$. Moreover, due to the symmetry of the normal distribution : $m_i = -m_{n+1-i}$.

The covariance matrix of the order statistics The order statistics $Z_{(1)}, Z_{(2)}, \ldots, Z_{(n)}$ are not independent. Therefore, we define their covariance matrix as :

$$V = \text{Cov} \begin{pmatrix} Z_{(1)} \\ Z_{(2)} \\ \vdots \\ Z_{(n)} \end{pmatrix}.$$

In other words : $V_{ij} = \text{Cov}\left(Z_{(i)}, Z_{(j)}\right)$.
Based on the vector m and the matrix V, the coefficients a_i are constructed in order to assign appropriate weights to the different ordered observations. A theoretical expression, depending on the convention used for the vectors, is :

$$a = \frac{m^\top V^{-1}}{\sqrt{m^\top V^{-1} V^{-1} m}}.$$

This construction determines the weights that make the comparison between the observations and the structure expected under normality as effective as possible.

Why are the a_i not calculated by hand ? In practice, the coefficients a_i are generally not calculated directly by hand. Indeed, one would first need to determine $E[Z_{(i)}]$ as well as the covariances $\mathrm{Cov}\left(Z_{(i)}, Z_{(j)}\right)$. These calculations quickly become complex as n increases.
Statistical software therefore uses numerical methods and specific approximations to obtain the coefficients required for the test.
For example, with Python and the `scipy` library, we can simply use : **scipy.stats.shapiro**().
The software automatically calculates the statistic W as well as the p-value.
Therefore, it is mainly important to understand the role of the coefficients a_i, rather than trying to calculate them manually. We can summarize this as follows :
The a_i weight the ordered observations in order to compare their structure with that expected under a normal distribution.
Intuitively :

$$W = \frac{\text{structure compatible with normality}}{\text{total variability}}.$$

— When the data are close to what is expected under normality, W is close to 1.
— When the data exhibit a substantial deviation from normality, W tends to decrease.

Statistical decision Once the statistic W has been calculated, the test provides a p-value. For a significance level of : $\alpha = 5\% = 0.05$, we apply the following rule :

$$\boxed{\begin{cases} p\text{-value} < 0.05 & \Rightarrow \text{reject } H_0, \\ p\text{-value} \geq 0.05 & \Rightarrow \text{do not reject } H_0. \end{cases}}$$

Thus, $p < 0.05$ means that the data provide sufficient evidence to conclude that they are not compatible with a normal distribution at the 5% significance level.
Conversely, $p \geq 0.05$ means that there is not sufficient evidence to reject normality.
Important : failing to reject H_0 does not mean that we have proved that the data are normally distributed. The correct formulation is :

"We do not reject the normality hypothesis."

and not :

"The data are normally distributed."

Python implementation The `scipy.stats` library allows us to directly perform the Shapiro–Wilk test.

Shapiro–Wilk test

```python
import numpy as np
from scipy.stats import shapiro

# Example of returns
returns = np.array([
    0.012, -0.008, 0.005, 0.003, -0.011,
    0.007, -0.004, 0.009, -0.002, 0.006
])

# Shapiro-Wilk test
W, p_value = shapiro(returns)

print("Shapiro-Wilk statistic:", W)
print("p-value:", p_value)

# Significance level
alpha = 0.05

if p_value < alpha:
    print("Reject H0: the returns are not normally distributed.")
else:
    print("Do not reject H0: the returns are consistent with normality.")
```

The result contains two main pieces of information :
— W : the test statistic ;
— the p-value : used to make the statistical decision.

Comparison between Shapiro–Wilk and Jarque–Bera Both tests have the same null hypothesis :

$$H_0 : \quad X \sim \mathcal{N}(\mu, \sigma^2).$$

However, they do not test normality in exactly the same way.

	Jarque–Bera	**Shapiro–Wilk**
Null hypothesis	The data follow a normal distribution	The data follow a normal distribution
Statistic	JB	W
Main idea	Uses skewness and kurtosis	Compares the structure of the ordered observations with that expected under normality
Characteristic value	$S = 0,\ K = 3$ under normality	W close to 1 under normality
Decision	Based on the p-value	Based on the p-value
If $p < 0.05$	Reject normality	Reject normality

TABLE 2.1 – Comparison between the Jarque–Bera and Shapiro–Wilk tests.

Joint interpretation of the two tests It can be useful to apply both tests to the same returns. If $p_{\mathrm{SW}} > 0.05$ and $p_{\mathrm{JB}} > 0.05$, neither test provides sufficient evidence to reject the normality hypothesis at the 5% significance level.
We can therefore conclude :

Normality is not rejected by either test at the 5% significance level.

Conversely, if $p_{\mathrm{SW}} < 0.05$ and $p_{\mathrm{JB}} < 0.05$, both tests reject the normality hypothesis. It is then reasonable to conclude that the returns exhibit a statistically significant deviation from a normal distribution.

When the two tests lead to different conclusions It is possible that the two tests do not lead to the same conclusion. For example, $p_{\mathrm{SW}} < 0.05$ but $p_{\mathrm{JB}} > 0.05$. This means that Shapiro–Wilk rejects normality whereas Jarque–Bera does not reject it.
This difference is not necessarily contradictory because the two tests use different information :

Shapiro–Wilk $\longrightarrow$ overall structure of the ordered observations

whereas :

Jarque–Bera $\longrightarrow$ skewness + kurtosis.

A difference in conclusions can therefore occur when the deviation from normality comes from a particular characteristic of the distribution, for example :

— substantial skewness ;
— heavier tails than those of a normal distribution ;
— extreme observations ;
— another form of deviation from normality.

$\Rightarrow$ The two tests are therefore complementary rather than competing.

Remark For financial returns, normality is an important assumption in many classical models. However, financial returns frequently exhibit skewness, kurtosis greater than 3, and fat tails. It is therefore particularly useful to combine several tools in order to obtain a more complete view of the distribution of returns.

2.5 Regulatory Risk Measures : VaR and CVaR

In financial risk management, knowing the average return of a portfolio is not sufficient : it is also essential to quantify the magnitude of potential losses, particularly under adverse and extreme scenarios. **Value-at-Risk (VaR)** and **Conditional Value-at-Risk (CVaR)**, also known as *Expected Shortfall (ES)*, are two fundamental measures used to quantify this risk.
VaR answers the following question :

> *What level of loss should the portfolio exceed only with a given probability, over a given horizon ?*

Concrete Example. Suppose a portfolio has a **95% VaR over a one-day horizon of** $50,000$.
The term "1 day" means that we study the portfolio's loss over a single day. The 95% confidence level means that we consider the quantile corresponding to the worst 5% of scenarios.
Mathematically, if $L_{t,1j}$ denotes the portfolio loss over one day, then :

$$\boxed{\mathbb{P}(L_{t,1j} > 50,000) = 5\%} \tag{2.18}$$

or equivalently :

$$\boxed{\mathbb{P}(L_{t,1j} \leq 50,000) = 95\%} \tag{2.19}$$

In other words, for a given day, the model estimates a 5% probability that the loss exceeds $50,000$. It is important to distinguish the different elements of this definition :

$$\boxed{\underbrace{1 \text{ day}}_{\text{horizon}} \qquad \underbrace{95\%}_{\text{confidence level}} \qquad \underbrace{5\%}_{\text{exceedance probability}}} \tag{2.20}$$

VaR therefore only provides a **loss threshold**. It does not specify the magnitude of the loss once this threshold is exceeded. This is precisely the role of **CVaR**. CVaR measures the **average loss in scenarios where VaR is exceeded**. It therefore answers the following question :

When the loss exceeds the VaR threshold, what is the average magnitude of that loss ?

For example, if the one-day 95% VaR is $50,000$ and the one-day 95% CVaR is $75,000$, this means that among the scenarios belonging to the worst 5% loss tail, the average loss is $75,000$.
CVaR therefore focuses on the portion of the distribution beyond the VaR threshold. Thus, the two measures provide complementary information :

$$\boxed{\text{VaR} \longrightarrow \text{threshold separating ordinary scenarios from extreme scenarios}} \tag{2.21}$$

$$\boxed{\text{CVaR} \longrightarrow \text{average loss in extreme scenarios beyond VaR}} \tag{2.22}$$

This distinction is particularly important in quantitative finance. Two portfolios may have the same VaR while having very different CVaRs. The portfolio with the higher CVaR is then exposed to potentially larger extreme losses.

Definition 2.5.1 *In general, consider a random variable L representing the **loss** of a portfolio over a given time horizon. For a confidence level $\alpha \in (0,1)$,*

— *the **Value-at-Risk** is defined as the α-quantile of the loss distribution :*

$$P(L > VaR_\alpha) = \alpha \tag{2.23}$$

— **Expected Shortfall (CVaR) :** *which measures the expected loss (absolute value), conditional on the loss exceeding the VaR threshold, is defined by*

$$CVaR_\alpha = E[L \mid L > VaR_\alpha] \tag{2.24}$$

Python Toolbox for Risk Calculation : To implement our three risk models (Historical, Parametric, Cornish-Fisher), we use the following key functions :

— `np.percentile(df, q)` : Extracts the q-th percentile of the distribution. For a historical VaR with a risk threshold $\alpha = 5\%$, we seek to isolate the breakpoint corresponding to the worst 5% of returns in the time series.

— `stats.norm.ppf(q)` : *Percent Point Function* (or quantile function). It is the inverse of the cumulative distribution function of the normal distribution. For $\alpha = 0.05$, it returns the standardized score $z_\alpha \approx -1.6448$.

— `df[df <= threshold].mean()` : Uses Pandas Boolean masking to isolate only returns less than or equal to the VaR quantile, and then computes their average. This is the direct implementation of the conditional expectation of historical Expected Shortfall.

```
# Definition of risk parameters
alpha = 0.05 # 5% threshold (95% confidence)
capital = 1000000 # $1,000,000 portfolio

# Calculation of empirical parameters
mu = log_returns.mean()
sigma = log_returns.std()

# --- METHOD 1: HISTORICAL (NON-PARAMETRIC) ---
# Extraction of the 5th empirical percentile
var_hist_ret = np.percentile(log_returns, alpha * 100)
var_historical = -var_hist_ret * capital

# Historical CVaR: average of returns below the VaR
cvar_hist_ret = log_returns[log_returns <= var_hist_ret].mean()
cvar_historical = -cvar_hist_ret * capital

# --- METHOD 2: GAUSSIAN PARAMETRIC ---
# Retrieval of the classical z-score via stats.norm.ppf
z_alpha = stats.norm.ppf(alpha)
var_parametric = -(mu + z_alpha * sigma) * capital

# Gaussian CVaR:
# Theoretical average of returns located in the left tail
# For alpha = 0.05, phi(z_alpha) / alpha corresponds to the expectation
# of a standard normal variable conditioned on Z <= z_alpha
cvar_parametric_ret = -(mu - sigma * stats.norm.pdf(z_alpha) / alpha)
cvar_parametric = cvar_parametric_ret * capital

# --- METHOD 3: CORNISH-FISHER (NON-GAUSSIAN ADJUSTMENT) ---
S = stats.skew(log_returns)
K = stats.kurtosis(log_returns, fisher=True)

# Polynomial extension of the z_alpha
z_cf = (z_alpha +
        (z_alpha**2 - 1) * S / 6 +
        (z_alpha**3 - 3 * z_alpha) * K / 24 -
        (2 * z_alpha**3 - 5 * z_alpha) * (S**2) / 36)

var_cornish_fisher = -(mu + z_cf * sigma) * capital

# Cornish-Fisher CVaR:
# Cornish-Fisher directly provides an approximation of the quantile.
# To obtain an approximation of CVaR, we integrate the Cornish-Fisher
# quantiles over the entire left tail of the distribution.
u_values = np.linspace(0.0001, alpha, 1000)

z_values = stats.norm.ppf(u_values)

z_cf_values = (z_values +
               (z_values**2 - 1) * S / 6 +
               (z_values**3 - 3 * z_values) * K / 24 -
               (2 * z_values**3 - 5 * z_values) * (S**2) / 36)
```

```
54
55 q_cf_values = mu + sigma * z_cf_values
56
57 # Numerical integration of the left-tail quantiles
58 cvar_cf_ret = np.trapezoid(q_cf_values, u_values) / alpha
59 cvar_cornish_fisher = -cvar_cf_ret * capital
60
61 print("\n--- Risk Analysis (1-Day Horizon, $1M Capital) ---")
62 print(f"Historical VaR (95\%)          : {var_historical:.2f} $")
63 print(f"Historical CVaR (95\%)         : {cvar_historical:.2f} $")
64 print(f"Gaussian VaR (95\%)            : {var_parametric:.2f} $")
65 print(f"Gaussian CVaR (95\%)           : {cvar_parametric:.2f} $")
66 print(f"Cornish-Fisher VaR (95\%)      : {var_cornish_fisher:.2f} $")
67 print(f"Cornish-Fisher CVaR (95\%)     : {cvar_cornish_fisher:.2f} $")
```

Listing 2.2 – Comparative Implementation of VaR and CVaR

Mathematical Origin of the Cornish-Fisher Expansion

The **Cornish-Fisher** formula is derived from the **Edgeworth expansion**, which approximates the cumulative distribution function of a non-normal distribution using the normal distribution. We seek the quantile q_α satisfying :

$$F(q_\alpha) = \alpha, \text{where F is the cumulative distribution function of a standard normal distribution}$$

We then express this quantile as a correction of the normal quantile z_α :

$$q_\alpha = z_\alpha + \delta,$$

where δ represents a correction related to the non-normal characteristics of the distribution.

By expanding the Edgeworth expansion around z_α and then approximately inverting the relationship, we obtain the Cornish-Fisher expansion :

$$\boxed{q_\alpha \approx z_\alpha + \frac{S}{6}(z_\alpha^2 - 1) + \frac{K}{24}(z_\alpha^3 - 3z_\alpha)\frac{S^2}{36}(2z_\alpha^3 - 5z_\alpha)} \tag{2.25}$$

where :

$$z_\alpha = \text{normal distribution quantile}, \qquad S = \text{skewness}, \qquad K = \text{excess kurtosis}. \tag{2.26}$$

The formula therefore corrects the normal quantile z_α to account for the **asymmetry** of the distribution, represented by S, and the **thickness of its tails**, represented by K.
When the distribution is normal :

$$S = 0 \qquad \text{and} \qquad K = 0. \tag{2.27}$$

The formula then reduces to :

$$\boxed{q_\alpha = z_\alpha} \tag{2.28}$$

The terms $z_\alpha^2 - 1$ and $z_\alpha^3 - 3z_\alpha$ naturally appear in the Edgeworth expansion and are related to the **Hermite polynomials** associated with the normal distribution.

2.5.1 Mathematical Interpretation of Cornish-Fisher CVaR

The important difference between VaR and CVaR is that the Cornish-Fisher formula directly provides an approximation of a **quantile**, and therefore of VaR, but not directly of CVaR.
The Cornish-Fisher VaR is approximately given by :

$$\boxed{\text{VaR}_\alpha^{CF} \approx -q_\alpha^{CF}} \tag{2.29}$$

where q_α^{CF} represents the return quantile obtained using the Cornish-Fisher expansion.
CVaR, however, requires considering all quantiles located in the left tail of the distribution. It corresponds to the average loss associated with the most unfavorable scenarios.
For a variable representing returns, CVaR can therefore be approximated by :

$$\boxed{\text{CVaR}_\alpha^{CF} \approx -\frac{1}{\alpha}\int_0^\alpha q_u^{CF}\,du} \tag{2.30}$$

where q_u^{CF} denotes the Cornish-Fisher quantile corresponding to the probability level u.
In our example, with $\alpha = 0.05$, we therefore consider the quantiles between 0 and 5% :

$$\text{CVaR}_{0.05}^{CF} \approx -\frac{1}{0.05}\int_0^{0.05} q_u^{CF}\,du. \tag{2.31}$$

The idea therefore differs from VaR. VaR uses a single quantile :

$$\text{VaR}_\alpha^{CF} \approx -q_\alpha^{CF}, \tag{2.32}$$

whereas CVaR uses all the quantiles in the tail :

$$\text{CVaR}_\alpha^{CF} \approx -\frac{1}{\alpha}\int_0^\alpha q_u^{CF}\,du. \tag{2.33}$$

In Python, the integral is approximated numerically by constructing a grid of probabilities :

$$u_1, u_2, \ldots, u_M \in [0, \alpha]. \tag{2.34}$$

We then calculate the corresponding Cornish-Fisher quantile for each value u_i :

$$q_{u_i}^{CF} = \mu + \sigma z_{CF}(u_i). \tag{2.35}$$

Finally, the integral is approximated numerically using the trapezoidal rule :

$$\int_0^\alpha q_u^{CF}\,du \approx \text{Trapz}\left(q_{u_1}^{CF}, \ldots, q_{u_M}^{CF}\right). \tag{2.36}$$

This provides an approximation of CVaR that accounts for the skewness and excess kurtosis of returns through the Cornish-Fisher expansion.
Thus :

$$\boxed{\text{Cornish-Fisher} \rightarrow \text{corrected quantiles} \rightarrow \text{tail integration} \rightarrow \text{CVaR}} \tag{2.37}$$

Chapitre 3

Modern Portfolio Theory & Optimization

3.0.1 Introduction to the Markowitz Framework

Published in 1952 by Harry Markowitz, *Modern Portfolio Theory* (MPT) laid the scientific foundations of diversification. Markowitz's fundamental paradigm is that a financial asset should not be analyzed in isolation, but according to its contribution to the overall risk-return trade-off of the portfolio to which it belongs.
The investor seeks to construct an optimal portfolio by combining assets whose imperfect correlation makes it possible to reduce overall risk (variance) without proportionally sacrificing expected return.

The Mathematical Framework Let us consider an investment universe composed of N risky financial assets. We define the vector of weights W allocated to each asset as :

$$W = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{pmatrix}$$

For a portfolio with no short selling and fully invested, we impose the following structural constraints :

$$\sum_{i=1}^{N} w_i = 1 \quad \text{and} \quad w_i \geq 0 \quad \forall i \in \{1, \ldots, N\}$$

Let $\mu_i = E(R_i)$ be the expected return of asset i, and μ the column vector of these expected returns. The expected return of the overall portfolio, denoted $E(R_p)$, is the mathematical expectation of the weighted sum of returns :

$$E(R_p) = \sum_{i=1}^{N} w_i E(R_i) = W^T \mu$$

Let $\Sigma \in \mathbb{R}^{N \times N}$ be the variance-covariance matrix of asset returns. The generic element σ_{ij} represents the covariance between asset i and asset j (if $i = j$, $\sigma_{ii} = \sigma_i^2$ is the variance of asset i). The overall portfolio variance σ_p^2 can be expressed as a quadratic form :

$$\sigma_p^2 = \sum_{i=1}^{N} \sum_{j=1}^{N} w_i w_j \sigma_{ij} = W^T \Sigma W$$

Thus, the portfolio standard deviation (or volatility) is

$$\sigma_p = \sqrt{W^T \Sigma W}$$

The objective of portfolio optimization is to determine how to allocate capital among several assets in order to obtain the best possible trade-off between return and risk.
Two classical formulations can be used to approach this problem. The first consists of finding, for a given level of return, the portfolio with the minimum risk. The second consists of directly finding the portfolio that maximizes the excess return obtained per unit of risk.
These two approaches are closely related and make it possible to understand the construction of the efficient frontier and the identification of the tangency portfolio.

3.0.2 Formulation 1 : Risk Minimization under a Target Return Constraint

A first way to construct the efficient frontier is to fix a target expected return level μ_0 and search, among all portfolios capable of achieving this return, for the one with the lowest risk.

Let

$$W = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{pmatrix}$$

be the vector of the weights of the different assets, μ the vector of expected returns, and Σ the covariance matrix of returns.
The optimization problem therefore consists of minimizing this variance for a fixed target return μ_0 :

$$(1) \quad \begin{aligned} \min_{W} \quad & \frac{1}{2} W^T \Sigma W \\ \text{subject to :} \quad & W^T \mu = \mu_0, \\ & W^T \mathbf{1} = 1, \\ & W \geq 0. \end{aligned}$$

The factor $\frac{1}{2}$ does not modify the optimal solution. It is traditionally introduced in order to simplify the derivatives of the objective function.

The different constraints have a simple economic interpretation :

— $W^T \mu = \mu_0$ imposes that the portfolio achieves exactly the target expected return ;

— $W^T \mathbf{1} = 1$ imposes that the entire capital is invested ;

— $W \geq 0$ prohibits short selling : each weight must be positive or zero.

For a given value of μ_0, the optimization therefore provides the portfolio with **minimum variance** among all portfolios with the target return.
By progressively varying the target return μ_0, we obtain a sequence of optimal portfolios. Each portfolio is characterized by a pair :

$$(\sigma_p, \mu_p).$$

The representation of these pairs in the return-risk plane then makes it possible to construct the **efficient frontier**.

Graphical Interpretation The following graph schematically illustrates the construction of the efficient frontier.
The region represents the set of feasible portfolios. Each portfolio is characterized by its risk σ_p and expected return μ_p. For a target return μ_0, we search for the feasible portfolio with the lowest risk.
The resulting point belongs to the efficient frontier. By repeating this optimization for different target return levels, we progressively construct the efficient frontier.

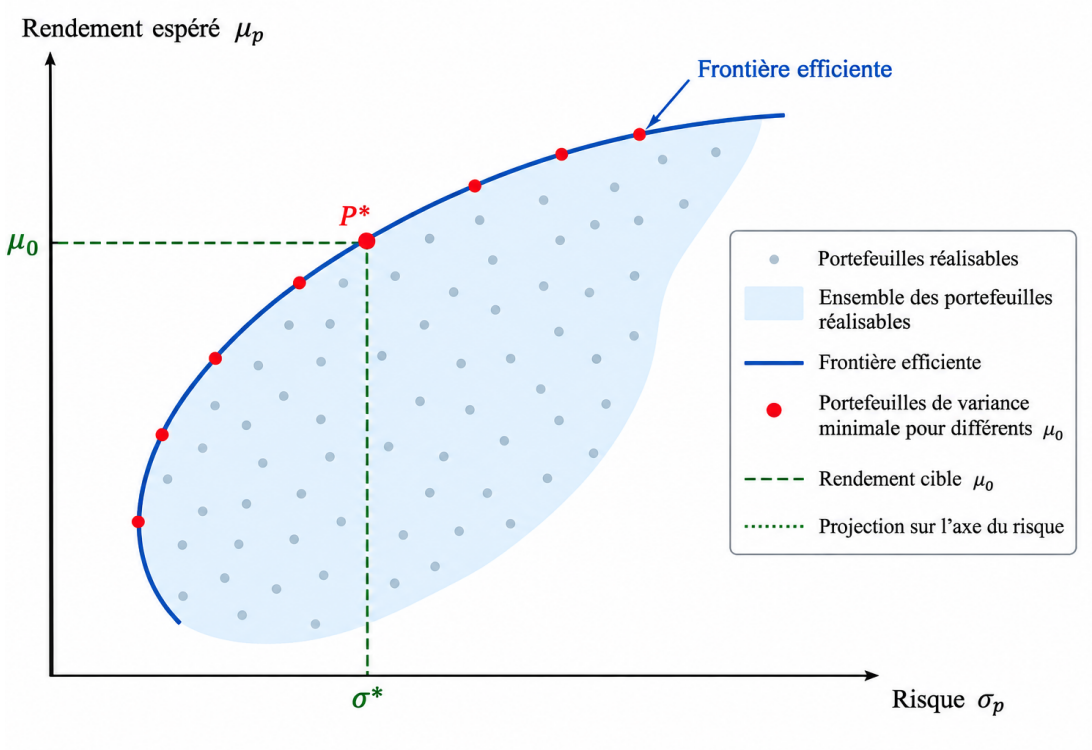


FIGURE 3.1 – Illustration of a negatively skewed distribution.

The principle is therefore as follows :

Target return	$\longrightarrow$	Variance minimization	$\longrightarrow$	Optimal portfolio

By repeating this optimization for different values of μ_0, we obtain the efficient frontier.

3.0.3 Formulation 2 : Sharpe Ratio Maximization

When a risk-free asset offering a certain return R_f is introduced, it becomes possible to compare risky portfolios while taking into account both their return and their level of risk.
The **Sharpe Ratio** measures the excess return obtained per unit of risk :

$$SR(W) = \frac{W^T\mu - R_f}{\sqrt{W^T\Sigma W}}$$

where :
— $W^T\mu$ represents the expected return of the risky portfolio ;
— R_f represents the return of the risk-free asset ;
— $\sqrt{W^T\Sigma W} = \sigma_p$ represents the volatility of the portfolio.

The objective is then to find the risky portfolio that maximizes the Sharpe Ratio :

$$\boxed{\begin{array}{rl} \max\limits_{W} & \dfrac{W^T\mu - R_f}{\sqrt{W^T\Sigma W}} \\ \text{subject to :} & W^T\mathbf{1} = 1, \\ & W \geq 0. \end{array}}$$

In practice, many numerical optimization algorithms, including those available in `scipy.optimize`, are formulated as minimization problems.
It is therefore sufficient to change the sign of the objective function :

$$\boxed{\min_{W} \quad -SR(W) = -\frac{W^T\mu - R_f}{\sqrt{W^T\Sigma W}}}$$

subject to the same constraints :

$$W^T\mathbf{1} = 1, \qquad W \geq 0.$$

The Tangency Portfolio : The solution to this problem is called the **tangency portfolio**, or **maximum Sharpe portfolio**.
To understand its graphical importance, let us place the risk-free asset in the return-risk plane.
A risk-free asset has zero volatility. It is therefore represented by the point $(0, R_f)$.
Now consider a risky portfolio P, characterized by (σ_P, μ_P). The line connecting the risk-free asset to portfolio P has slope :

$$\frac{\mu_P - R_f}{\sigma_P}.$$

But this expression corresponds exactly to the Sharpe Ratio of the portfolio :

$$\boxed{\text{slope of the line} = SR(P)}$$

Thus, maximizing the Sharpe Ratio graphically means searching, among all lines originating from the risk-free asset, for the one with the highest slope.
The portfolio corresponding to the point of tangency with the efficient frontier is called the **tangency portfolio**.
The line passing through the risk-free asset and the tangency portfolio is called the **Capital Market Line** (CML). Its slope is equal to the maximum Sharpe Ratio :

$$\boxed{\text{slope of the CML} = SR(W^*) = \max_{W} SR(W)}$$

where W^* denotes the tangency portfolio.

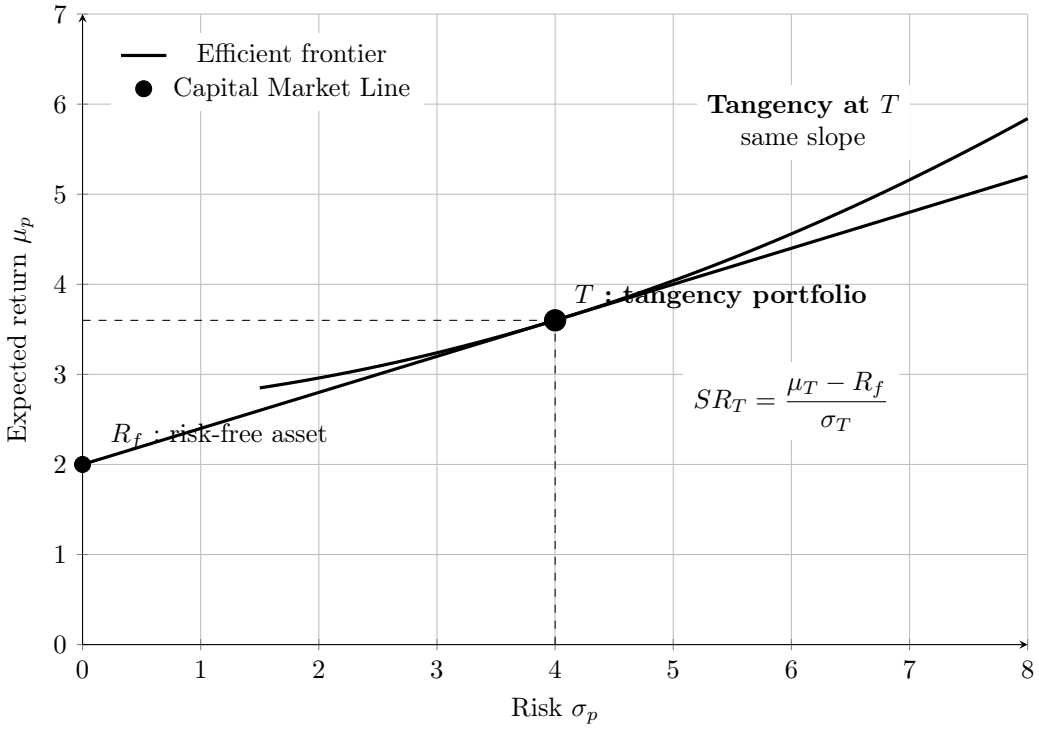


FIGURE 3.2 – The tangency portfolio T is the point where the Capital Market Line is tangent to the efficient frontier. The slope of this line is equal to the Sharpe Ratio of the tangency portfolio. The data are purely illustrative.

Combination of the Risk-Free Asset and the Tangency Portfolio

Once the tangency portfolio W^* has been determined, the investor can combine this portfolio with the risk-free asset. If α ($0 \leq \alpha \leq 1$) denotes the proportion invested in the tangency portfolio, then the proportion invested in the risk-free asset is $1 - \alpha$ and the expected return of the overall portfolio is :

$$\mu_C = (1 - \alpha)R_f + \alpha\mu_{W^*}.$$

We can also write :

$$\boxed{\mu_C = R_f + \alpha(\mu_{W^*} - R_f)}$$

and its risk is :

$$\boxed{\sigma_C = \alpha\sigma_{W^*}.}$$

These two equations show that the portfolios obtained by combining the risk-free asset and the tangency portfolio lie on a straight line in the return-risk plane : the **Capital Market Line**.

3.0.4 Formulation 3 : The Sharpe Ratio along the Efficient Frontier

The two previous formulations can be connected through a third representation that is particularly useful for understanding the role of the tangency portfolio.
For each target return μ_0, the first optimization provides a minimum-variance portfolio $W(\mu_0)$. We can then calculate its Sharpe Ratio :

$$SR(\mu_0) = \frac{\mu_0 - R_f}{\sigma_p(\mu_0)}$$

with :

$$\sigma_p(\mu_0) = \sqrt{W(\mu_0)^T \Sigma W(\mu_0)}.$$

We therefore obtain a function that associates each target return with the Sharpe Ratio of the corresponding efficient portfolio :

$$\boxed{\mu_0 \quad \longmapsto \quad SR(\mu_0)}$$

The tangency portfolio then corresponds to the point at which this function reaches its maximum :

$$\boxed{\mu_0^* = \underset{\mu_0}{\text{argmax}}\, SR(\mu_0)}$$

In other words, the tangency portfolio is the efficient portfolio that offers the best excess return per unit of risk.

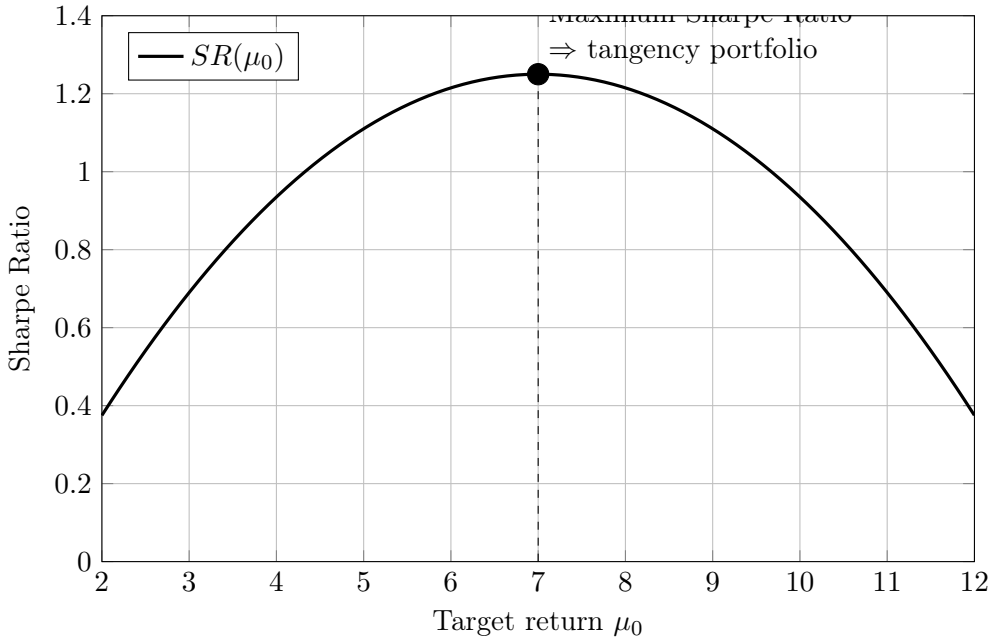


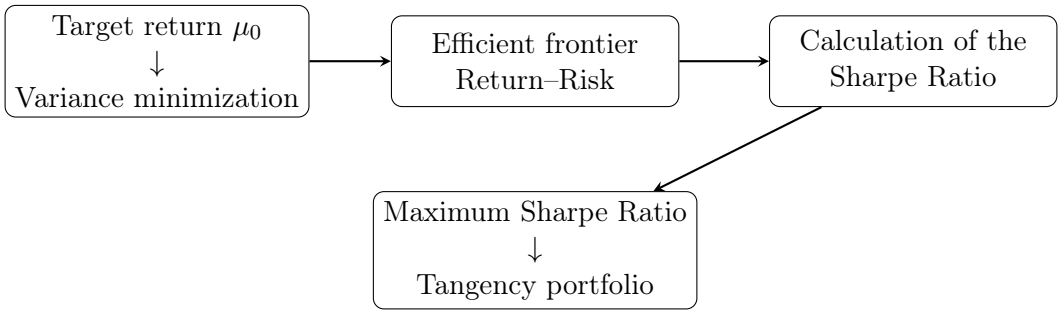
FIGURE 3.3 – Evolution of the Sharpe Ratio along the efficient frontier. The maximum corresponds to the tangency portfolio. The values shown are purely illustrative.

This third graph directly highlights the link between the two optimization problems.

For each value of μ_0, we have an efficient portfolio. We then calculate its Sharpe Ratio. The maximum of the curve corresponds to the tangency portfolio.

3.0.5 Graphical Summary

The three representations make it possible to progressively understand the reasoning behind portfolio optimization.



The fundamental idea to remember is therefore :

Minimize risk for each target return
$\Longrightarrow$ Construct the efficient frontier
Maximize the Sharpe Ratio
$\Longrightarrow$ Identify the tangency portfolio

The first formulation therefore makes it possible to construct the efficient frontier, while the second makes it possible to select, on this frontier, the portfolio that offers the best excess return per unit of risk when a risk-free asset is available.

3.0.6 SLSQP Method — Sequential Least Squares Programming

SLSQP is a constrained optimization method based on the principle of *Sequential Quadratic Programming* (SQP).
We seek to solve the problem

$$\min_x f(x) \qquad \text{subject to} \qquad g(x) = 0, \quad h(x) \geq 0. \tag{3.1}$$

Principle We start from an **initial point** x_0, chosen by the user. At iteration k, rather than directly searching for the new x, we seek a **displacement** d from the current point :

$$\boxed{x = x_k + d}. \tag{3.2}$$

We then locally approximate the objective using a second-order Taylor expansion :

$$f(x_k + d) \simeq f(x_k) + \nabla f(x_k)^T d + \frac{1}{2} d^T H_k d. \tag{3.3}$$

Since $f(x_k)$ is constant with respect to d, the optimal displacement is obtained by solving :

$$\begin{aligned} \min_d \quad & \nabla f(x_k)^T d + \frac{1}{2} d^T H_k d \\ \text{subject to} \quad & g(x_k) + \nabla g(x_k)^T d = 0, \\ & h(x_k) + \nabla h(x_k)^T d \geq 0. \end{aligned} \tag{3.4}$$

The constraints are here **linearized** around x_k, while the objective is approximated by a quadratic function. The resulting problem is therefore a **quadratic programming (QP) problem**. The solution d_k provides a direction of displacement. The new iterate is then :

$$\boxed{x_{k+1} = x_k + \alpha_k d_k}, \tag{3.5}$$

where α_k is determined by a line search in order to control the length of the displacement.

Role of H_k : H_k represents an **approximation of the Hessian of the Lagrangian**

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \lambda^T g(x) - \mu^T h(x). \tag{3.6}$$

It therefore contains information about the **local curvature** of the problem. In practice, H_0 is often initialized with the identity matrix :

$$H_0 = I, \tag{3.7}$$

and then updated during the iterations using a quasi-Newton method, typically **BFGS**.

Link with Gradient Descent : Gradient descent uses a first-order approximation :

$$f(x_k + d) \simeq f(x_k) + \nabla f(x_k)^T d, \tag{3.8}$$

which leads to the direction :

$$d_k = -\nabla f(x_k). \tag{3.9}$$

SLSQP generalizes this idea to constrained problems : it searches for the **best local displacement** d while taking into account both the **curvature** of the objective through H_k and the **constraints** through their linearization.
Thus :

$$\boxed{\text{Gradient descent : } d_k = -\nabla f(x_k)} \tag{3.10}$$

whereas SLSQP searches for :

$$\boxed{d_k = \arg\min_d \left[\nabla f(x_k)^T d + \frac{1}{2} d^T H_k d \right] \quad \text{subject to linearized constraints.}} \tag{3.11}$$

3.0.7 Some Remarks

In financial *Data Engineering* practice, raw data extracted from markets (via yFinance, for example) are captured at a **daily** frequency. However, performance, management mandates, and asset comparisons are universally expressed over an **annual** horizon. It is therefore essential to annualize the calculated statistics.

A standard trading year contains approximately **252 trading days** (excluding weekends and public holidays).

— Let R_t be the daily log-return of an asset. Under the assumption of stationarity, the expected daily return is $\mu_d = E(R_t)$. The annual log-return is obtained by summing the daily log-returns over the 252 trading days, so the expectation follows by linearity :

$$\mu_{\text{annuel}} = E\left(\sum_{t=1}^{252} R_t \right) = 252 \times E(R_t) = \mu_d \times 252$$

— Under the standard assumption of independent and identically distributed (i.i.d.) daily returns (absence of temporal autocorrelation, a property related to weak-form market efficiency), the variance of a sum of random variables is equal to the sum of their variances :

$$\sigma^2_{\text{annuel}} = \text{Var}\left(\sum_{t=1}^{252} R_t \right) = \sum_{t=1}^{252} \text{Var}(R_t) = 252 \times \sigma_d^2$$

Therefore, for volatility (standard deviation), the relationship follows a **square-root-of-time** rule :

$$\sigma_{\text{annuel}} = \sqrt{252 \times \sigma_d^2} = \sigma_d \times \sqrt{252}$$

In Python code, directly multiplying the daily variance-covariance matrix `returns.cov()` by 252 makes it possible to annualize all variances (diagonal elements) and covariances (off-diagonal elements) at once according to this statistical rule :

$$\Sigma_{\text{annuel}} = 252 \times \Sigma_{\text{journalière}}.$$

3.0.8 Complete Scripted Implementation

Here is the clean and commented implementation of the complete script in Python :

```
# ============================================================
# 0. IMPORTING LIBRARIES
# ============================================================
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
import scipy.optimize as sco

# ============================================================
# 1. DATA EXTRACTION AND CLEANING
# ============================================================

# List of assets composing our investment universe
assets = ['AAPL', 'MSFT', 'AMZN', 'NVDA']

# Retrieval of adjusted closing prices from Yahoo Finance
# for the period from 01/01/2021 to 01/01/2026.
# Adjusted prices account in particular for splits and dividends.
data = yf.download(
    assets,
    start='2021-01-01',
    end='2026-01-01'
)['Close']

# ------------------------------------------------------------
# Calculation of daily logarithmic returns
# ------------------------------------------------------------
# data.shift(1) shifts prices by one day:
#
#             P_t
# r_t = ln( -------- )
#             P_{t-1}
#
# dropna() removes the first observation because there is no
# P_{t-1} for the first day.
returns = np.log(data / data.shift(1)).dropna()

# ============================================================
# 2. ANNUALIZATION OF STATISTICS
# ============================================================
# We consider approximately 252 trading days per year.
TRADING_DAYS = 252

# ------------------------------------------------------------
# Annualized average return
```

```
# ------------------------------------------------------------

# returns.mean() calculates the average daily logarithmic return
# for each asset.
# We annualize by multiplying by 252:
# mu_i = 252 * average(r_i)
mean_returns = returns.mean() * TRADING_DAYS

# ------------------------------------------------------------
# Annualized covariance matrix
# ------------------------------------------------------------
# returns.cov() calculates the daily covariance matrix:
#
#                        Cov(r_1,r_1)   Cov(r_1,r_2) ...
# Sigma_daily = ...
# To annualize the covariance:
# Sigma = 252 * Sigma_daily
#
cov_matrix = returns.cov() * TRADING_DAYS


# ============================================================
# 3. MONTE CARLO SIMULATION
# ============================================================

# Number of random portfolios we will generate
num_portfolios = 10000

# We create a matrix of dimension 3 x 10,000.
# For each portfolio, we will store:
# results[0,i] = portfolio volatility
# results[1,i] = portfolio return
# results[2,i] = Sharpe Ratio
#
results = np.zeros((3, num_portfolios))

# This list will store the weights
# of each of the 10,000 portfolios.
weights_record = []

# Fixing the random seed to obtain
# the same results at each execution.
np.random.seed(42)

# ------------------------------------------------------------
# Generation of random portfolios
# ------------------------------------------------------------

for i in range(num_portfolios):

    # Generation of 4 positive random numbers,
    # one for each of the 4 assets.
    weights = np.random.random(len(assets))

    # ---------------------------------------------------------
    # Weight normalization
    # ---------------------------------------------------------
    # We impose:
    #
    # w_1 + w_2 + ... + w_N = 1
    # that is, 100% of the capital is invested.
```

```
    weights /= np.sum(weights)

    # We store the portfolio weights
    weights_record.append(weights)

    # -------------------------------------------------------
    # Portfolio return
    # -------------------------------------------------------
    # Mathematical formula:
    # mu_p = w^T mu
    #
    # that is:
    # mu_p = sum(w_i * mu_i)
    #
    p_ret = np.dot(weights, mean_returns)

    # -------------------------------------------------------
    # Portfolio volatility
    # -------------------------------------------------------
    # Portfolio variance:
    # sigma_p^2 = w^T Sigma w
    #
    # Volatility:
    # sigma_p = sqrt(w^T Sigma w)
    #
    p_vol = np.sqrt(
        np.dot(
            weights.T,
            np.dot(cov_matrix, weights)
        )
    )

    # -------------------------------------------------------
    # Sharpe Ratio
    # -------------------------------------------------------
    # General formula:
    #
    # Sharpe = (mu_p - R_f) / sigma_p
    #
    # Here, we assume R_f = 0.
    # Therefore:
    # Sharpe = mu_p / sigma_p
    #
    p_sharpe = p_ret / p_vol

    # Storing the results
    results[0, i] = p_vol
    results[1, i] = p_ret
    results[2, i] = p_sharpe

# ============================================================
# 4. NUMERICAL OPTIMIZATION
# ============================================================
# We define a function that calculates the three statistics
# of a portfolio from its weights.
def portfolio_stats(weights):

    # Transformation into a NumPy array
    weights = np.array(weights)

    # -------------------------------------------------------
```

```
    # Portfolio return
    # ------------------------------------------------------
    # mu_p = w^T mu
    #
    p_ret = np.dot(weights, mean_returns)

    # ------------------------------------------------------
    # Volatility
    # ------------------------------------------------------
    # sigma_p = sqrt(w^T Sigma w)
    #
    p_vol = np.sqrt(
        np.dot(
            weights.T,
            np.dot(cov_matrix, weights)
        )
    )

    # ------------------------------------------------------
    # Sharpe Ratio
    # ------------------------------------------------------
    # R_f = 0
    #
    # Sharpe = mu_p / sigma_p
    #
    p_sharpe = p_ret / p_vol

    # We return:
    # [return, volatility, Sharpe]
    return np.array([p_ret, p_vol, p_sharpe])

# ------------------------------------------------------------
# Optimization objective function
# ------------------------------------------------------------

def min_func_sharpe(weights):

    # We want to MAXIMIZE the Sharpe Ratio.
    # But scipy.optimize.minimize() seeks to MINIMIZE
    # a function.
    # Therefore:
    # maximize(Sharpe)
    # is equivalent to:
    # minimize(-Sharpe)
    #
    return -portfolio_stats(weights)[2]

# ============================================================
# OPTIMIZATION CONSTRAINTS
# ============================================================

# ------------------------------------------------------------
# Constraint: the sum of weights must equal 1
# ------------------------------------------------------------
# scipy requires a constraint of the form:
# f(x) = 0
# We therefore define:
# sum(w_i) - 1 = 0
# which is equivalent to:
# sum(w_i) = 1
#
```

```
230 constraints = (
231     {
232         'type': 'eq',
233         'fun': lambda x: np.sum(x) - 1
234     }
235 )
236 
237 # ------------------------------------------------------------
238 # Weight constraints
239 # ------------------------------------------------------------
240 # Each weight must satisfy:
241 # 0 <= w_i <= 1
242 # This means:
243 # - no short selling
244 # - no asset can have a weight greater than 100%
245 bounds = tuple(
246     (0, 1) for asset in range(len(assets))
247 )
248 
249 # ------------------------------------------------------------
250 # Algorithm starting point
251 # ------------------------------------------------------------
252 # We start with an equally weighted portfolio:
253 #
254 # w = (25%, 25%, 25%, 25%)
255 #
256 initial_guess = len(assets) * [1. / len(assets)]
257 
258 # ============================================================
259 # SOLVING THE OPTIMIZATION PROBLEM
260 # ============================================================
261 
262 # We seek the portfolio that minimizes:
263 #
264 #          - mu_p
265 # min =  -----------
266 #         sigma_p
267 #
268 # which is equivalent to maximizing the Sharpe Ratio:
269 #
270 #              mu_p
271 # max=  -----------
272 #            sigma_p
273 #
274 # subject to:
275 # sum(w_i) = 1
276 # and:
277 # 0 <= w_i <= 1
278 #
279 opt_results = sco.minimize(
280     min_func_sharpe,
281     initial_guess,
282     method='SLSQP',
283     bounds=bounds,
284     constraints=constraints
285 )
286 
287 # Retrieval of the optimal weights
288 optimal_weights = opt_results.x
289 
290 # ============================================================
```

```
# 5. PORTFOLIO VISUALIZATION
# ============================================================

plt.figure(figsize=(10, 6))

# ------------------------------------------------------------
# Scatter plot of the 10,000 simulated portfolios
# ------------------------------------------------------------
# X-axis:
# portfolio volatility = risk
# Y-axis:
# expected portfolio return
# Color:
# Sharpe Ratio
#
plt.scatter(
    results[0, :],        # Volatility
    results[1, :],        # Return
    c=results[2, :],      # Sharpe Ratio
    cmap='viridis',
    marker='o',
    s=10,
    alpha=0.3
)

# Color bar indicating the Sharpe Ratio value
plt.colorbar(
    label='Annualized Sharpe Ratio'
)

# ------------------------------------------------------------
# Addition of the optimal portfolio
# ------------------------------------------------------------

# portfolio_stats(optimal_weights) returns:
#
# [return, volatility, Sharpe]
# Therefore:
# [1] = volatility
# [0] = return
#
plt.scatter(
    portfolio_stats(optimal_weights)[1],
    portfolio_stats(optimal_weights)[0],
    color='red',
    marker='*',
    s=200,
    label='Max Sharpe (Optimum)'
)

# Titles and axes of the graph
plt.title(
    "Markowitz Portfolio \& Optimal Portfolio",
    fontsize=12
)

plt.xlabel(
    "Annualized Volatility (Risk)"
)

plt.ylabel(
```

```
    "Annualized Expected Return"
)

# Legend
plt.legend()

# Grid
plt.grid(
    True,
    linestyle='--',
    alpha=0.5
)

# Saving the graph
plt.savefig(
    'frontiere_efficiente.png'
)

# Closing the figure
plt.close()

# ============================================================
# 6. DISPLAYING THE RESULTS
# ============================================================

print("-" * 40)
print("OPTIMAL ALLOCATION (MAX SHARPE):")
print("-" * 40)

# We associate each asset with its optimal weight.
for asset, weight in zip(assets, optimal_weights):

    # :.2\% displays the weight as a
    # percentage with two decimal places.
    print(
        f"Optimal weight for {asset} : {weight:.2\%}"
    )

print("-" * 40)
```

3.1 Machine-Based Practical Work (Exercises)

Exercise 1 : The Minimum Variance Portfolio (Global Minimum Variance - GMV)

1. Write a Python function `min_func_variance(weights)` that returns only the calculated portfolio volatility.
2. Use `scipy.optimize.minimize` with the `SLSQP` algorithm, the same bounds and constraints, to extract the weights of the GMV portfolio.
3. Add this point to the previous graph (use a blue square marker, for example) to distinguish it from the Max Sharpe portfolio.

3.1.1 Solution

1. Place after `min_func_sharpe()` :

```
# ------------------------------------------------------------
# Objective function: Global Minimum Variance Portfolio (GMV)
# ------------------------------------------------------------

def min_func_variance(weights):

    # We seek to MINIMIZE the portfolio volatility.
    #
    # portfolio_stats(weights) returns:
    # [return, volatility, Sharpe]
    #
    # We therefore retrieve only the volatility:
    return portfolio_stats(weights)[1]

```

2. Just after the optimization of the Max Sharpe portfolio :

```
# Retrieval of the optimal weights
optimal_weights = opt_results.x


# ============================================================
# OPTIMIZATION OF THE GMV PORTFOLIO
# ============================================================

# We now seek to directly minimize
# the portfolio volatility.
#
# Constraints:
#
# sum(w_i) = 1
#
# and:
#
# 0 <= w_i <= 1

gmv_results = sco.minimize(
    min_func_variance,
    initial_guess,
    method='SLSQP',
    bounds=bounds,
    constraints=constraints
)
```

```
27
28
29 # Retrieval of the optimal weights of the GMV portfolio
30 gmv_weights = gmv_results.x
```

`gmv_weights` contains the weights of the Global Minimum Variance Portfolio.

3. After the Max Sharpe portfolio, add the following code :

```
1  # ------------------------------------------------------------
2  # Addition of the GMV portfolio
3  # ------------------------------------------------------------
4
5  # portfolio_stats(gmv_weights) returns:
6  #
7  # [return, volatility, Sharpe]
8  #
9  # Therefore:
10 #
11 # [1] = volatility
12 # [0] = return
13
14 plt.scatter(
15     portfolio_stats(gmv_weights)[1],
16     portfolio_stats(gmv_weights)[0],
17     color='blue',
18     marker='s',
19     s=100,
20     label='GMV (Minimum Variance)'
21 )
```

Exercise 2 : Integration of Allocation Limits (Regulatory Weight Constraints)

1. Modify the `bounds` argument of the optimizer so that no individual asset can represent more than 40% of the total portfolio, and none can fall below the minimum threshold of 5%.
2. Solve the Max Sharpe problem again under this geometric constraint.
3. Verify with a `print` that all calculated weights strictly satisfy :

$$0.05 \leq w_i \leq 0.40 \quad \forall i$$

Solution

1. Replace,

```
1 bounds = tuple(
2     (0, 1) for asset in range(len(assets))
3 )
```

with

```
1 bounds = tuple(
2     (0.05, 0.40) for asset in range(len(assets))
3 )
```

2. Run the Max Sharpe optimization again

```
1 # ============================================================
2 # SOLVING THE MAX SHARPE PROBLEM UNDER CONSTRAINTS
3 # ============================================================
```

```
4
5  opt_results = sco.minimize(
6      min_func_sharpe,
7      initial_guess,
8      method='SLSQP',
9      bounds=bounds,
10     constraints=constraints
11 )
12
13 optimal_weights = opt_results.x
```

3. Add the following code :

```
1  # ============================================================
2  # CONSTRAINT VERIFICATION
3  # ============================================================
4
5  print("-" * 40)
6  print("CONSTRAINT VERIFICATION:")
7  print("-" * 40)
8
9  for asset, weight in zip(assets, optimal_weights):
10
11     print(
12         f"{asset} : {weight:.2\%} "
13         f"| Satisfies bounds : {0.05 <= weight <= 0.40}"
14     )
15
16 print("-" * 40)
17
18 print(
19     "All weights satisfy the constraints:",
20     np.all((optimal_weights >= 0.05) &
21            (optimal_weights <= 0.40))
22 )
```

Exercise 3 – How Stable Is a Markowitz Portfolio ?

Consider the following six assets :

$$\text{AAPL}, \quad \text{MSFT}, \quad \text{AMZN}, \quad \text{NVDA}, \quad \text{JPM}, \quad \text{XOM}.$$

Use daily adjusted closing prices over the last five years and Assume :

$$w_i \geq 0, \qquad \sum_{i=1}^{6} w_i = 1, \qquad R_f = 0.$$

A – Maximum Sharpe Ratio Portfolio

1. Compute daily logarithmic returns :

$$r_{i,t} = \ln\left(\frac{P_{i,t}}{P_{i,t-1}}\right).$$

2. Annualize the expected returns and covariance matrix :

$$\mu_{\text{annuel}} = 252\,\mu_{\text{daily}},$$

$$\Sigma_{\text{annuel}} = 252\,\Sigma_{\text{daily}}.$$

3. Using the SLSQP method, determine the portfolio maximizing :

$$\max_{w} \frac{w^\top \mu}{\sqrt{w^\top \Sigma w}}$$

subject to :

$$\sum_{i=1}^{6} w_i = 1, \qquad w_i \geq 0.$$

4. Report :
   — the optimal weights ;
   — the expected return ;
   — the volatility ;
   — the Sharpe ratio.

B – Does the Estimation Period Matter ?

1. Repeat the optimization using three different estimation windows :

12 months	36 months	60 months

   For each window, compute the maximum-Sharpe portfolio.
2. Present the weights in the following table :

	AAPL	*MSFT*	*AMZN*	*NVDA*	*JPM*	*XOM*
12 months						
36 months						
60 months						

   Then create a graph comparing the three allocations.

C – Interpretation

Answer the following questions.

1. Question 1.
   — Are the three optimal portfolios similar or different ?
   — Which assets have the most stable weights ?
   — Which assets have the largest changes ?
2. Question 2.
   — Why can changing the estimation period lead to different optimal portfolios ?
   — Discuss the role of :

$$\mu \qquad \text{and} \qquad \Sigma.$$

3. Question 3.
   Suppose that two estimation periods produce very different portfolios. Does this necessarily mean that the Markowitz model is wrong ? Explain your answer.

4. Question 4.
   In practice, why might an investor prefer to impose additional allocation constraints such as

$$0.05 \leq w_i \leq 0.30$$

   instead of using the unconstrained optimal portfolio ?

Chapitre 4

Asset Pricing Models (CAPM) and Factor Analysis

4.1 Introduction

In the previous chapter, we saw how to construct a portfolio by optimizing the return-risk trade-off using the Markowitz approach. However, a fundamental question remains : **how can we assess whether the return of an individual asset or portfolio is justified given the risk taken ?**

This chapter formalizes the transition from a purely mathematical approach (Markowitz) to a structural approach to performance. We will first study the Capital Asset Pricing Model (CAPM) to decompose risk into systematic and idiosyncratic components. We will then extend this framework to multi-factor models (Fama-French) before seeing how to move away from rigid economic theories through a purely statistical method : Principal Component Analysis (PCA).

The market portfolio : It is a theoretical portfolio that represents the entire financial market. It contains all assets available in the market, with each asset weighted according to its relative market value.
For a publicly traded company, the **market capitalization** is defined as :

$$\text{Capitalisation}_i = P_i \times N_i,$$

where :

— P_i represents the price of one share of company i ;
— N_i represents the total number of shares outstanding.

Market capitalization therefore represents the total value of the company in the market. It should not be confused with the price of a single share. The weight of asset i in the market portfolio is then given by :

$$w_i^M = \frac{\text{Capitalisation}_i}{\sum_{j=1}^{n} \text{Capitalisation}_j}.$$

We can therefore verify that :

$$\sum_{i=1}^{n} w_i^M = 1.$$

For example, if the total market capitalization is 10 billion euros and a company has a capitalization of 1 billion euros, its weight in the market portfolio is :

$$w_i^M = \frac{1}{10} = 10\%.$$

Thus, the market portfolio does not assign greater weight to assets with the highest expected return. The weights are determined by the relative size of companies in the market.

Link with the tangency portfolio : The tangency portfolio is obtained within the framework of mean-variance optimization. When a risk-free asset is available, it corresponds to the risky portfolio that maximizes the Sharpe ratio :

$$\max_w \frac{w^\top \mu - R_f}{\sqrt{w^\top \Sigma w}}.$$

The tangency portfolio therefore depends on μ, Σ and R_f.
The market portfolio and the tangency portfolio are therefore, in general, two different concepts :

$$\boxed{\text{Market portfolio} \quad \neq \quad \text{Tangency portfolio}}$$

The market portfolio is constructed from market capitalizations, whereas the tangency portfolio is obtained through an optimization procedure.
However, within the theoretical framework of the **Capital Asset Pricing Model (CAPM)**, under its assumptions, these two portfolios coincide :

$$\boxed{w^M = w^T}$$

where w^M represents the market portfolio and w^T the tangency portfolio.
This equality is a central result of the CAPM. In this theoretical framework, all investors hold the same risky portfolio, namely the market portfolio. They differ only in the proportion invested in the risk-free asset.
The relationship between the two concepts can therefore be summarized as follows :

Market portfolio capitalization-weighted	$\overset{\text{CAPM Assumptions}}{=}$	Tangency portfolio maximizing the Sharpe ratio

In practice, the tangency portfolio calculated from historical data does not necessarily coincide with the observed market portfolio. The equality between the two is a theoretical result of the CAPM and relies on its assumptions.

4.2 The Mathematical Framework

4.2.1 The CAPM (Capital Asset Pricing Model)

The CAPM (or MEDAF) states that the expected return of a risky asset i, denoted $E(R_i)$, is determined solely by its exposure to systematic market risk. The fundamental formula is :

$$E(R_i) = R_f + \beta_i \left(E(R_m) - R_f\right).$$

Where :

— R_f is the risk-free rate.

— $E(R_m)$ is the expected return of the market portfolio (e.g., S&P 500 or CAC 40).
— $(E(R_m) - R_f)$ represents the market risk premium.
— β_i measures the sensitivity and contribution of the asset to market risk.

The coefficient β_i is mathematically defined as the ratio of the covariance between the asset's return and the market return, divided by the market variance :

$$\beta_i = \frac{\text{Cov}(R_i, R_m)}{\text{Var}(R_m)}.$$

Linear Regression and Performance Decomposition (α and β) In financial practice, we estimate this model through a standard linear regression (*Ordinary Least Squares - OLS*) on historical data :

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i(R_{m,t} - R_{f,t}) + \epsilon_{i,t}.$$

Where :

— α_i (Jensen's Alpha) represents the abnormal excess return of the asset. A positive and statistically significant alpha indicates a skilled portfolio manager or an exploitable market anomaly.
— $\epsilon_{i,t}$ is the error term, representing the risk specific to the company (e.g., a scandal or a technological success specific to the company) ; in this model, it is generally assumed that $E(\epsilon_{i,t}) = 0$. This risk can be eliminated through diversification, unlike systematic risk (β_i).

4.2.2 Multi-Factor Models : The Fama-French Three-Factor Model

Given the empirical limitations of the single-factor CAPM, Eugene Fama and Kenneth French (1993) introduced two additional macroeconomic risk factors to explain returns :

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i(R_{m,t} - R_{f,t}) + s_i\mathbf{SMB}_t + h_i\mathbf{HML}_t + \epsilon_{i,t}.$$

Where :

— **SMB** (*Small Minus Big*) : the SMB factor represents the **size premium**. It measures the difference in return between a portfolio composed of small-capitalization companies (*Small*) and a portfolio composed of large-capitalization companies (*Big*).
In simplified form, the SMB factor can be represented as :

$$SMB = R_{\text{Small}} - R_{\text{Big}}.$$

A positive value of SMB means that, over the period considered, small-cap stocks achieved a higher return than large-cap stocks. Conversely, a negative value indicates that large-cap stocks achieved a higher return.
The existence of a difference in returns between small and large companies constitutes an important empirical observation in the study of financial markets. This difference is often referred to as the **size premium**. It may notably be interpreted as compensation associated with certain characteristics and risks specific to small companies, such as lower liquidity, greater sensitivity to financing conditions, or greater vulnerability to economic conditions.
It is nevertheless important to distinguish this empirical observation from its economic interpretation : a positive size premium does not necessarily imply that the outperformance of small-cap stocks is exclusively compensation for illiquidity risk.

— **HML** (*High Minus Low*) : the HML factor represents the **value premium**. It measures the difference in return between companies with a high *Book-to-Market* ratio (*High*) and those with a low *Book-to-Market* ratio (*Low*).
In simplified form :

$$HML = R_{\text{High B/M}} - R_{\text{Low B/M}}.$$

The *Book-to-Market* ratio compares a company's book value of equity with its market value :

$$\text{Book-to-Market} = \frac{\text{Book value of shareholders' equity (Assets-debts)}}{\text{Market capitalization}}.$$

Companies with a high *Book-to-Market* ratio are generally classified as *Value* companies. Their book value is relatively large compared with their market valuation.
Conversely, companies with a low *Book-to-Market* ratio are generally associated with the *Growth* style. Their market valuation is relatively high compared with their book value.
A positive value of HML means that, over the period considered, *Value* companies achieved a higher return than *Growth* companies. A negative value corresponds to the opposite situation.
The HML factor therefore makes it possible to measure a portfolio's exposure to the value factor. Positive exposure to the HML factor corresponds to a more *Value*-oriented position, whereas negative exposure corresponds more closely to a *Growth* orientation. "'

4.2.3 Non-Parametric Approach : Principal Component Analysis (PCA)

Unlike factor approaches that impose factors defined a priori by economic theory (for example, Market, Size, or Style), PCA is a descriptive statistical method used to extract the main sources of common variation from a large set of assets.
The first step is to start from asset prices and calculate their returns :

$$R_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}},$$

where $P_{i,t}$ denotes the price of asset i at date t.
We then obtain a return matrix containing the time-series observations for all assets. From these returns, we construct the covariance matrix :

$$\Sigma_{ij} = \text{Cov}(R_i, R_j).$$

The matrix Σ therefore contains information about how the returns of different assets move together. It is precisely this covariance structure that PCA seeks to summarize.
Mathematically, PCA consists of diagonalizing the covariance matrix by finding its eigenvalues λ_k and associated eigenvectors V_k :

$$\Sigma V_k = \lambda_k V_k.$$

Each eigenvector defines a particular direction in return space. This direction makes it possible to construct a linear combination of returns, called a **principal component** :

$$PC_k = V_k^\top R,$$

where R denotes the vector of asset returns.

Thus, principal components are themselves factors constructed from the observed returns. For example, the first principal component can be written as :

$$PC_1 = v_{11}R_1 + v_{21}R_2 + \cdots + v_{N1}R_N.$$

The coefficients v_{i1} are called *loadings* and indicate the contribution of each asset to the construction of the factor.
The eigenvalues λ_k measure the amount of variance explained by each component. The proportion of variance explained by component k is given by :

$$\frac{\lambda_k}{\sum_{j=1}^{N} \lambda_j}.$$

When the first principal component has loadings with the same sign and comparable magnitude across many assets, it can be interpreted empirically as a factor representing a common market movement.
However, it is important to distinguish this statistical factor from the theoretical market portfolio. PCA does not know the economic weights of the market and does not directly construct the market portfolio. It simply seeks the directions that explain the greatest amount of variance in the observed returns.
The approach can therefore be summarized as follows :

$$\boxed{\text{Prices} \rightarrow \text{Returns} \rightarrow \text{Covariances} \rightarrow \text{PCA} \rightarrow \text{Statistical Factors}}$$

4.2.4 Some Remarks

The main objective of PCA is not to directly predict asset returns, but to **summarize the information contained in a large number of returns through a reduced number of statistical factors**.
Suppose that we observe the returns of N assets :

$$R_t = \begin{pmatrix} R_{1,t} \\ R_{2,t} \\ \vdots \\ R_{N,t} \end{pmatrix}.$$

When assets move in a similar manner, their returns exhibit significant correlations and covariances. A large part of the variation observed in the N returns can then be explained by a few common movements.
PCA seeks precisely to identify these common movements. It therefore makes it possible to move from a large number of observed variables to a reduced number of factors :

$$\boxed{R_1, R_2, \ldots, R_N \quad \longrightarrow \quad PC_1, PC_2, \ldots, PC_K} \qquad K \ll N.$$

For example, instead of analyzing separately the returns of 100 assets, PCA may make it possible to summarize a large part of their variation through only a few principal components.
The first principal component is the linear combination of returns that explains the largest possible share of the variance. The second explains the largest remaining share of variance, subject to being orthogonal to the first, and so on.
The first components can therefore be interpreted as the main **statistical factors** present in the data. For example, if the coefficients of PC_1 are positive and relatively similar for a large number of stocks, this component may represent a common movement of the entire market.

However, it is important to distinguish this approach from that of the CAPM or the Fama-French model. In these models, the factors are defined **a priori** based on an economic theory :

$$\begin{aligned} \text{CAPM} &\rightarrow \text{Market Factor,} \\ \text{Fama-French} &\rightarrow \text{Market, SMB, HML.} \end{aligned}$$

With PCA, by contrast, the factors are not imposed in advance. They are **extracted directly from the data** based on the covariance structure of returns :

Returns → Covariance Matrix → PCA → Statistical Factors

PCA therefore constitutes a complementary approach to economic factor models. It makes it possible to empirically search for the main sources of common variation without assuming their economic interpretation in advance.
Once these statistical factors have been identified, it is also possible to use them in a linear regression model. For example, with the first three principal components :

$$R_{i,t} = \alpha_i + \beta_{i1} PC_{1,t} + \beta_{i2} PC_{2,t} + \beta_{i3} PC_{3,t} + \varepsilon_{i,t}.$$

In this approach, PCA is therefore first used to **construct the factors**, while regression is then used to **measure each asset's exposure to these factors** through the coefficients β_{ik}.
Thus, the two steps answer two different questions :

PCA :	What are the main common factors present in the data ?
Regression :	How is each asset exposed to these factors ?

4.3 Python Implementation

4.3.1 Key Functions

statsmodels.api.add_constant(X) : By default, the linear regression function of the **statsmodels** library fits a line that necessarily passes through the origin $(0, 0)$. Mathematically, this would amount to forcing α to be zero from a statistical perspective ($\alpha = 0$).

To correctly estimate the equation $Y = \alpha + \beta X$, a column filled with 1s must be added to our matrix of independent variables X. This is the role of `add_constant`. Without this function, the calculated β coefficient would be biased because all residual excess performance of the asset would be artificially incorporated into the slope of the line.

sm.OLS(Y, X).fit() : This method implements the Ordinary Least Squares (OLS) estimator. The objective is to estimate the model coefficients by minimizing the sum of squared residuals. For each observation, the residual is the difference between the observed value and the value predicted by the model :

$$\epsilon_t = Y_t - \hat{Y}_t$$

OLS therefore seeks the coefficients that minimize :

$$\min \sum_{t=1}^{T} \epsilon_t^2$$

Graphically, this corresponds to finding the regression line that minimizes the sum of the squared vertical distances between the observations and the estimated line.

In the CAPM framework, we can write :

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i(R_{M,t} - R_{f,t}) + \epsilon_t$$

The `.fit()` method then estimates α and β ($\alpha = model.params[0]$ and $\beta = model.params[1]$) and returns various statistics used to evaluate the regression, including *t-statistics*, *p-values*, and R^2.

The t-statistic (*t-stat*) : The t-statistic measures how far an estimated coefficient is from zero relative to the uncertainty of the estimate :

$$t = \frac{\hat{\theta}}{SE(\hat{\theta})}$$

For example, if :

$$\hat{\beta} = 1.20 \qquad \text{and} \qquad SE(\hat{\beta}) = 0.10$$

then :

$$t = \frac{1.20}{0.10} = 12$$

A large absolute value of t indicates that the coefficient is relatively far from zero compared with its estimation uncertainty. The associated *p-value* can then be used to test the hypothesis $H_0 : \beta = 0$.

The coefficient of determination R^2 : The R^2 measures the proportion of the variability in Y explained by the model :

$$R^2 = 1 - \frac{\sum_{t=1}^{T}(Y_t - \hat{Y}_t)^2}{\sum_{t=1}^{T}(Y_t - \bar{Y})^2}$$

For example, if $R^2 = 0.64$, the model explains 64% of the observed variability in Y in the sample, while the remaining 36% is not explained by the variables included in the model.

Thus, in a CAPM regression :

- $\hat{\alpha}$ and $\hat{\beta}$ are the estimated coefficients ;
- the *t-stat* measures their distance from zero relative to their uncertainty ;
- the *p-value* is used to test whether they are statistically different from zero ;
- R^2 indicates the proportion of return variability explained by the model.

PCA(n_components=k) From the `sklearn.decomposition` package, this class configures the dimensionality reduction algorithm. The `n_components` argument defines the number of eigenvectors that we wish to retain.

The execution takes place in three phases automated by the `.fit()` function :

1. Internal calculation of the covariance matrix of the asset basket.
2. Singular Value Decomposition (*Singular Value Decomposition-SVD*) to extract the orthogonal axes maximizing residual variance.
3. Ordering of the components in decreasing order of importance (from the highest eigenvalue to the lowest).

pca.components This attribute stores a matrix containing the **factor loadings**. Each row corresponds to a factor (PC1, PC2...) and each column contains the asset's attribution coefficient within that factor.

If all the values in the first row (`pca.components_[0]`) have the same sign and similar magnitude, this validates the financial interpretation : the first factor acts as an **implicit market index** that drives all stocks in the same direction.

Example of Interpreting `pca.components_` : To better understand the interpretation of the factors obtained through Principal Component Analysis (PCA), consider a portfolio composed of three stocks : Apple, Microsoft, and Tesla.
Suppose that we have daily returns for these three stocks and apply PCA while requesting two principal components :

```
pca = PCA(n_components=2)
pca.fit(returns)

print(pca.components_)
```

Suppose that Python returns the following matrix :

$$\texttt{pca.components_} = \begin{pmatrix} 0.60 & 0.55 & 0.58 \\ -0.70 & 0.10 & 0.70 \end{pmatrix}$$

The columns correspond to the different assets, while the rows correspond to the two principal components :

	Apple	Microsoft	Tesla
PC_1	0.60	0.55	0.58
PC_2	−0.70	0.10	0.70

Interpretation of the first component : The first row corresponds to PC_1. The three coefficients are positive and relatively close to one another. This means that the three stocks contribute in the same direction to this component.
This component can be represented, in a simplified way, as :

$$PC_{1,t} = 0.60R_{\text{Apple},t} + 0.55R_{\text{Microsoft},t} + 0.58R_{\text{Tesla},t}$$

The subscript t is important : it indicates that the stock returns change over time. Thus, the coefficients 0.60, 0.55, and 0.58 are determined by the PCA, while the returns $R_{\text{Apple},t}$, $R_{\text{Microsoft},t}$, and $R_{\text{Tesla},t}$ vary from one day to another.
Consequently, the value of the factor PC_1 also changes over time.

Example of the factor's evolution over time : Suppose that the observed returns over three days are :

	Apple	Microsoft	Tesla
Day 1	1.0%	0.8%	1.2%
Day 2	−0.5%	−0.3%	−0.7%
Day 3	2.0%	1.5%	2.3%

For each date, the value of the factor PC_1 is obtained by combining the observed returns with the PCA coefficients.
For example, for the first day :

$$PC_{1,1} = 0.60(1.0\%) + 0.55(0.8\%) + 0.58(1.2\%)$$

For the second day :

$$PC_{1,2} = 0.60(-0.5\%) + 0.55(-0.3\%) + 0.58(-0.7\%)$$

And for the third day :

$$PC_{1,3} = 0.60(2.0\%) + 0.55(1.5\%) + 0.58(2.3\%)$$

We therefore obtain a time series for the factor PC_1 :

Date	PC_1
Day 1	factor value on Day 1
Day 2	factor value on Day 2
Day 3	factor value on Day 3

Therefore, even though the coefficients in `pca.components_` remain fixed, the value of the factor PC_1 changes over time because the returns of the underlying assets change over time.

Interpretation of the second component The second row corresponds to PC_2 :

$$PC_{2,t} = -0.70R_{\text{Apple},t} + 0.10R_{\text{Microsoft},t} + 0.70R_{\text{Tesla},t}$$

Here, the coefficients associated with Apple and Tesla have opposite signs. Therefore, the component PC_2 can represent a contrast between the movements of these two assets.
Since the coefficient associated with Microsoft is close to zero, its contribution to this component is relatively small.
The following intuitive rule can be useful :

- coefficients with the same sign indicate that the assets contribute in the same direction to the factor ;
- coefficients with opposite signs indicate that the factor contrasts the movements of certain assets ;
- a coefficient close to zero indicates that the asset contributes relatively little to the component.

Distinguishing between factors and coefficients : It is important to distinguish between the *loadings* and the factors themselves.
In the previous example, the values 0.60, 0.55, and 0.58 are the coefficients (or *loadings*) associated with PC_1.
These coefficients indicate **how each asset contributes to the factor**. They do not represent the performance of the asset and do not change from one date to another when the PCA is estimated only once over the entire sample.
The factor PC_1, on the other hand, is a time series :

$$PC_1 = (PC_{1,1}, PC_{1,2}, PC_{1,3}, \ldots, PC_{1,T})$$

Each value corresponds to the value of the factor at a given date.
In Python, the factor values can be obtained using :

```
factors = pca.transform(returns)
```

They can then be organized into a table :

```
factor_df = pd.DataFrame(
    factors,
    index=returns.index,
    columns=["PC1", "PC2"]
)
```

This produces a structure such as :

Date	PC_1	PC_2
01/01	0.021	−0.013
02/01	−0.014	0.008
03/01	0.035	0.019
04/01	−0.006	−0.022

This table therefore represents the time evolution of the factors extracted by PCA. The distinction can be summarized as follows :

`pca.components_` $\longrightarrow$ composition of the factors

`pca.transform(returns)` $\longrightarrow$ factor values over time

In other words, `pca.components_` answers the question :

"How are the assets combined to construct the factor ?"

whereas `pca.transform(returns)` answers the question :

"What is the value of this factor at each date ?"

Evolution of the coefficients over time Finally, it is important to distinguish between two situations.
If PCA is estimated only once over the entire sample, the coefficients in `pca.components_` remain fixed. It is the value of the factors that changes over time.
However, if PCA is re-estimated regularly, for example every year or using a rolling window, the coefficients themselves may also change :

$$\text{PCA}_{t_1} \rightarrow \text{coefficients}_{t_1}$$

$$\text{PCA}_{t_2} \rightarrow \text{coefficients}_{t_2}$$

This makes it possible to study whether the factor structure of the portfolio changes over time.

4.3.2 Python Code

The following script retrieves data for an asset (`AAPL`) and its market index (`^GSPC` for the S&P 500) to estimate the CAPM using `statsmodels`, and then applies PCA using `scikit-learn` to a subset of assets.

```
import numpy as np
import pandas as pd
import yfinance as yf
import statsmodels.api as sm
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt


# ======================================================================
# 1. CAPM ESTIMATION VIA STATSMODELS
# ======================================================================

```

```
13 # ---------------------------------------------------------------------
14 # 1.1 Data retrieval
15 # ---------------------------------------------------------------------
16 # We select:
17 #   - AAPL: the asset we want to analyze
18 #   - ^GSPC: the S&P 500, used here as a proxy for the market portfolio
19 #
20 # The theoretical market portfolio contains all assets
21 # in the economy, weighted by their market capitalization.
22 #
23 # In practice, it is not directly observable.
24 # An equity index is therefore often used as an approximation.
25 #
26 # Here:
27 #
28 #       R_M = R_S&P500
29 #
30 # where R_M denotes the return of the market portfolio.
31
32 tickers = ['AAPL', '^GSPC']
33
34 data = yf.download(tickers, start='2021-01-01', end='2026-01-01')['Close']
35
36
37 # ---------------------------------------------------------------------
38 # 1.2 Calculation of daily returns
39 # ---------------------------------------------------------------------
40 # pct_change() calculates:
41 #
42 #       R_t = (P_t - P_{t-1}) / P_{t-1}
43 #
44 # and dropna() removes the first observation, for which a return
45 # cannot be calculated.
46
47 returns = data.pct_change().dropna()
48
49
50 # ---------------------------------------------------------------------
51 # 1.3 Definition of the CAPM variables
52 # ---------------------------------------------------------------------
53 # We seek to explain the return of AAPL stock
54 # using the market return.
55 #
56 # Dependent variable:
57 #
58 #       Y = R_AAPL
59 #
60 # Explanatory variable:
61 #
62 #       X = R_S&P500 = R_M
63 #
64 # Simplifying assumption:
65 #
66 #       R_f = 0
67 #
68 # The CAPM model is therefore written:
69 #
70 #       R_AAPL,t = alpha + beta * R_M,t + epsilon_t
71 #
72 # In a complete version with a risk-free rate:
73 #
```

```
#       R_AAPL,t - R_f,t = alpha + beta (R_M,t - R_f,t) + epsilon_t

Y = returns['AAPL']
X = returns['^GSPC']


# ----------------------------------------------------------------------
# 1.4 Adding the constant
# ----------------------------------------------------------------------
# statsmodels does not automatically add a constant.
#
# The constant corresponds here to the alpha of the model.
#
# We therefore obtain:
#
#       X_with_const = [1, R_M]
#
# which makes it possible to estimate:
# R_AAPL = alpha + beta * R_M + epsilon

X_with_const = sm.add_constant(X)


# ----------------------------------------------------------------------
# 1.5 Estimation using OLS
# ----------------------------------------------------------------------
# OLS = Ordinary Least Squares
#
# The model estimates the alpha and beta coefficients that minimize
# the sum of squared errors.

model = sm.OLS(Y, X_with_const).fit()

# Display the complete statistical summary
print(model.summary())


# ----------------------------------------------------------------------
# 1.6 Extraction of the main results
# ----------------------------------------------------------------------

# Daily alpha:
# constant estimated by the regression

alpha_journalier = model.params['const']

# Beta: sensitivity of AAPL's return to market variations.
#
# For example: beta = 1.2 means that, on average, a 1 \% change in
# the market is associated with a 1.2 \% change in AAPL,
# all else being equal.

beta_estime = model.params['^GSPC']


# ----------------------------------------------------------------------
# 1.7 Annualization of alpha
# ----------------------------------------------------------------------
# Alpha is estimated at a daily frequency.
#
# Here, we use an approximate linear annualization:
```

```
135
136 alpha_annuel = alpha_journalier * 252
137
138
139 # ----------------------------------------------------------------------
140 # 1.8 Display of the results
141 # ----------------------------------------------------------------------
142
143 print("\n--- CAPM RESULTS ---")
144
145 print(f"Beta (Systematic Risk) : {beta_estime:.4f}")
146
147 print(f"Annual Alpha (Performance) : {alpha_annuel:.2\%}")
148
149 print(
150     f"R-squared (Variance explained by the market) : "
151     f"{model.rsquared:.2\%}"
152 )
153
154
155 # ======================================================================
156 # 2. EXTRACTION OF RISK FACTORS VIA PCA
157 # ======================================================================
158
159 # ----------------------------------------------------------------------
160 # 2.1 Construction of a diversified basket of assets
161 # ----------------------------------------------------------------------
162 # Unlike the CAPM, we do not define a market portfolio
163 # in advance here.
164 #
165 # We simply provide PCA with a set of assets:
166 #
167 #       AAPL, MSFT, AMZN, NVDA, JPM, V, XOM, PG
168 #
169 # PCA will search for the main sources of common variation
170 # present in their returns.
171
172 basket = ['AAPL', 'MSFT','AMZN','NVDA','JPM','V','XOM', 'PG']
173
174
175 # ----------------------------------------------------------------------
176 # 2.2 Price retrieval
177 # ----------------------------------------------------------------------
178
179 basket_data = yf.download(
180     basket,
181     start='2021-01-01',
182     end='2026-01-01'
183 )['Close']
184
185
186 # ----------------------------------------------------------------------
187 # 2.3 Return calculation
188 # ----------------------------------------------------------------------
189
190 basket_returns = basket_data.pct_change().dropna()
191
192
193 # ----------------------------------------------------------------------
194 # 2.4 PCA
195 # ----------------------------------------------------------------------
```

```
# Here we ask PCA to extract the first 3 principal components.
#
# Each component is a linear combination of the returns
# of the different assets.
#
# For example, the first factor can be written:
#
#       PC1 = w1 R_AAPL + w2 R_MSFT + ... + w8 R_PG
#
# The coefficients w_i are called "loadings".
#
# IMPORTANT:
# sklearn's PCA centers the variables by default,
# meaning that it subtracts their mean.
#
# It does NOT automatically standardize the variables
# to unit variance.

pca = PCA(n_components=3)

# Estimation of the principal components

pca.fit(basket_returns)


# ------------------------------------------------------------------------
# 2.5 Explained variance
# ------------------------------------------------------------------------
# explained_variance_ratio_ indicates the proportion of total variance
# explained by each component.
#
# Example: PC1: 50 \%, PC2: 20 \% and PC3: 10 \%
# means that the first three factors together explain
# 80 \% of the total return variance.

explained_variance = pca.explained_variance_ratio_


# ------------------------------------------------------------------------
# 2.6 Display of explained variance
# ------------------------------------------------------------------------

print("\n--- PCA RESULTS ---")

for i, variance in enumerate(explained_variance):

    print(
        f"Principal Factor {i+1} "
        f"(PC{i+1}) explains : "
        f"{variance:.2\%} of total variance"
    )


# Total variance explained by the first 3 factors

print(
    f"Total variance explained by the first 3 factors : "
    f"{np.sum(explained_variance):.2\%}"
)


```

```
# ======================================================================
# 3. INTERPRETATION OF THE FIRST FACTOR: PC1
# ======================================================================

# ----------------------------------------------------------------------
# 3.1 Extraction of PC1 loadings
# ----------------------------------------------------------------------
# components_[0] contains the coefficients of the first principal component.
#
# Each coefficient indicates the contribution of the asset
# to the construction of PC1.

loadings_pc1 = pd.Series(
    pca.components_[0],
    index=basket)


# ----------------------------------------------------------------------
# 3.2 Display of the loadings
# ----------------------------------------------------------------------

print("\n--- PC1 LOADINGS ---")

print(loadings_pc1)


# ----------------------------------------------------------------------
# 3.3 Visualization of the loadings
# ----------------------------------------------------------------------
# The graph shows which assets contribute positively or negatively to PC1.

plt.figure(figsize=(8, 4))

loadings_pc1.plot(
    kind='bar',
    color='darkblue'
)

plt.title(
    "Factor Loadings of the First Principal Factor (PC1)"
)

plt.ylabel("Weight in the factor")

plt.axhline(
    0,
    color='black',
    linestyle='--',
    linewidth=0.8
)

plt.grid(
    axis='y',
    alpha=0.3
)

plt.tight_layout()

plt.savefig('pca_loadings.png')

plt.close()
```

```
318
319
320 # ======================================================================
321 # 4. LINK BETWEEN THE CAPM, THE MARKET PORTFOLIO AND PCA
322 # ======================================================================
323
324 # ----------------------------------------------------------------------
325 # CAPM
326 # ----------------------------------------------------------------------
327 #
328 # In the CAPM, the risk factor is defined in advance:
329 #
330 #       R_M = return of the market portfolio
331 #
332 # Since the true market portfolio is not directly observable,
333 # we use here: R_M = R_S&P500
334 #
335 # This is precisely the variable that appears in:
336 # X = returns['^GSPC']
337 #
338 # The model then estimates:
339 # R_AAPL = alpha + beta R_M + epsilon
340 #
341 # ----------------------------------------------------------------------
342 # PCA
343 # ----------------------------------------------------------------------
344 #
345 # In PCA, the factor is NOT defined in advance.
346 # The factors are constructed directly from the data.
347 # PCA searches for the directions that explain the most variance
348 # in asset returns.
349 #
350 # We obtain: PC1, PC2, PC3, ... which are statistical factors.
351 #
352 # PC1 may sometimes resemble a market factor if the different
353 # assets have loadings with the same sign and comparable magnitude.
354 #
355 # But: PC1 is automatically different from the market portfolio
356 # and the market portfolio is not explicitly used
357 # in this part of the code.
358
359
360 # ======================================================================
361 # 5. CONCEPTUAL SUMMARY
362 # ======================================================================
363
364 print("\n--- CONCEPTUAL SUMMARY ---")
365
366 print("""
367 CAPM:
368     Factor = market
369     Proxy used here = S&P 500 (^GSPC)
370     Objective = measure AAPL's exposure to market risk
371
372 PCA:
373     Factors = statistically constructed from returns
374     Factors extracted here = PC1, PC2 and PC3
375     Objective = identify the main common sources
376                 of asset variation
377
378 Market portfolio:
```

```
    It is NOT explicitly constructed as a vector of weights
    in this code.

    It intervenes indirectly in the CAPM through:
        R_M = R_S&P500

    On the other hand, it is not directly used in the PCA.
""")
```

4.4 Exercises

Exercise 1 : Linear Regression and Estimation of CAPM Parameters The objective of this exercise is to empirically estimate Jensen's α and the β of an investment fund relative to its benchmark index.

You must write a Python script that fulfills the following conditions :

1. Stochastically generates a historical series of 60 months of returns for the market portfolio and for the asset being studied (using a fixed random seed for reproducibility).
2. Calculates the excess returns of the asset and the market relative to a fixed risk-free rate.
3. Uses the `statsmodels` library to perform a linear regression using Ordinary Least Squares (OLS). *Warning : make sure to explicitly include a constant for the estimation of the intercept.*
4. Extracts and displays the estimated coefficients as well as the statistical significance (*p-value*) of α.

Python Solution Code

```python
import numpy as np
import pandas as pd
import statsmodels.api as sm

# 1. Generation of historical data (60 months)
np.random.seed(42)
rendements_marche = np.random.normal(0.006, 0.04, 60) # Average market return = 0.6\%/month
taux_sans_risque = 0.001                               # Rf = 0.1\% per month constant

# Asset generation (True beta = 1.3, Target monthly alpha = 0.16\%)
bruit = np.random.normal(0, 0.02, 60)
rendements_actif = taux_sans_risque + 1.3 * (rendements_marche - taux_sans_risque) + 0.0016 + bruit

# 2. Calculation of excess returns
exces_marche = rendements_marche - taux_sans_risque
exces_actif = rendements_actif - taux_sans_risque

# 3. Modeling and OLS Regression via statsmodels
# A constant must be added to estimate Jensen's Alpha
X = sm.add_constant(exces_marche)
modele = sm.OLS(exces_actif, X).fit()

# 4. Display of the report and extraction of coefficients
print(modele.summary())
print(f"\n--- Estimation Results ---")
```

```
print(f"Estimated Beta (slope) : {modele.params[1]:.4f}")
print(f"Estimated monthly Alpha (intercept) : {modele.params[0]:.6f}")
print(f"Alpha P-value : {modele.pvalues[0]:.4f}")
```

Exercise 2 : Factor Extraction through Principal Component Analysis (PCA) Within factor analysis, we often seek to identify unobservable (statistical) factors that drive the behavior of a basket of stocks.

Write a Python script that performs the following tasks :

1. Creates a DataFrame of historical returns for 4 distinct financial assets, constructed so that they share a strong correlation induced by an invisible common factor.
2. Performs standardization (centering and scaling) of the return data, a mathematically mandatory step before any PCA.
3. Applies the Principal Component Analysis algorithm from `scikit-learn` to extract the first two components.
4. Displays the percentage of total variance explained by the first principal component.
5. Extracts the *loadings* (weights of the first component) for each asset and interprets these results with respect to factor sensitivities.

Python Solution Code

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# 1. Generation of correlated returns for a portfolio of 4 assets
np.random.seed(10)
facteur_commun = np.random.normal(0, 0.02, 100)
rendements = pd.DataFrame({
    'Actif_A': 0.8 * facteur_commun + np.random.normal(0, 0.01, 100),
    'Actif_B': 1.2 * facteur_commun + np.random.normal(0, 0.01, 100),
    'Actif_C': 0.5 * facteur_commun + np.random.normal(0, 0.01, 100),
    'Actif_D': 1.0 * facteur_commun + np.random.normal(0, 0.01, 100)
})

# 2. Standardization of return data
scaler = StandardScaler()
rendements_scales = scaler.fit_transform(rendements)

# 3. Initialization and fitting of PCA
pca = PCA(n_components=2)
pca.fit(rendements_scales)

# 4. Variance explained by the first implicit market factor
variance_expliquee = pca.explained_variance_ratio_[0] * 100
print(f"Variance explained by Principal Component 1 : {variance_expliquee:.2f
    }\%\n")

# 5. Extraction of statistical betas (Loadings)
print("Factor sensitivities (PC1 Loadings) :")
for actif, loading in zip(rendements.columns, pca.components_[0]):
    print(f"  Sensitivity of {actif} : {loading:.4f}")
```

Exercise 3 — CAPM Estimation and Interpretation We want to study the relationship between the return of a stock and the market return using the CAPM market model.

We consider the stocks `AAPL`, `MSFT`, and `JPM`, as well as the S&P500 index `GSPC` as an approximation of the market portfolio.

1. Download the adjusted prices of the four assets over the period from January 1, 2021 to January 1, 2026.
2. Calculate the arithmetic daily returns.
3. For each of the three stocks, estimate the model :

$$R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t},$$

   where $R_{M,t}$ is the return of the S&P 500.
4. Use `statsmodels` and `OLS` to estimate α_i and β_i.
5. Economically interpret the coefficient β_i, α_i, and the R^2 of each regression.
6. Compare the three stocks. Which stock appears to have the greatest exposure to systematic market risk ?
7. Graphically verify the relationship between the returns of each stock and those of the S&P 500 using a scatter plot and the regression line.

Solution : We consider the stocks `AAPL`, `MSFT`, and `JPM`, as well as the S&P 500 index `^GSPC` as an approximation of the market portfolio.

1. Download the data We begin by downloading the adjusted prices :

```
import numpy as np
import pandas as pd
import yfinance as yf
import statsmodels.api as sm
import matplotlib.pyplot as plt

tickers = ['AAPL', 'MSFT', 'JPM', '^GSPC']

data = yf.download(
tickers,
start='2021-01-01',
end='2026-01-01'
)['Adj Close']

print(data.head())
```

“‘latex We then have a time series containing the adjusted prices of the three stocks and the S&P 500. The symbol `^GSPC` corresponds to the S&P 500. In this exercise, the S&P 500 is used as an empirical approximation of the market portfolio.

2. Calculating daily returns Daily arithmetic returns are calculated using :

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

In Python :

```
returns = data.pct_change().dropna()
print(returns.head())
```

The function `pct_change()` directly calculates R_t
The `dropna()` method then removes the first observation, for which no return can be calculated.

3. Estimating the CAPM models For each of the three stocks, we estimate the model :

$$R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t}.$$

In our case $R_{M,t} \approx R_{\text{S\&P500},t}$.
We can therefore estimate the three regressions separately.

```
assets = ['AAPL', 'MSFT', 'JPM']
results = {}
for asset in assets:
Y = returns[asset]
X = returns['^GSPC']
X_with_const = sm.add_constant(X)
model = sm.OLS(Y, X_with_const).fit()
results[asset] = model
print("\n==============================")
print(asset)
print("==============================")
print(model.summary())
```

The command `sm.add_constant(X)` adds a column of 1 to the explanatory variable in order to allow the estimation of alpha :

$$R_{i,t} = \alpha_i \times 1 + \beta_i R_{M,t} + \varepsilon_{i,t}.$$

The coefficient associated with the constant is therefore α_i, while the coefficient associated with the S&P 500 is β_i.

4. Interpreting β, α and R^2 : The main results can be extracted as follows :

```
for asset in assets:
model = results[asset]
alpha = model.params['const']
beta = model.params['^GSPC']
r_squared = model.rsquared
print(f"\n{asset}")
print(f"Daily alpha       : {alpha:.6f}")
print(f"Beta              : {beta:.4f}")
print(f"R-squared         : {r_squared:.2\%}")
```

Beta interpretation. The coefficient β_i measures the sensitivity of the stock's return to market movements :

$$\beta_i = \frac{\text{Cov}(R_i, R_M)}{\text{Var}(R_M)}.$$

For example, if a stock has $\beta_i = 1.2$, then, all else being equal, a 1% market movement is associated with an average movement of approximately 1.2% in the stock according to the model.
A β greater than 1 indicates a higher sensitivity to the market than the market itself.
A β lower than 1 indicates lower sensitivity.

Alpha interpretation. Alpha corresponds to the constant of the regression :

$$R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t}.$$

It represents the average component of the return that is not explained by the market factor in this regression.
The estimated alpha should be interpreted with caution : a non-zero alpha in the sample does not necessarily mean that it is statistically different from zero.

Interpretation of R^2. The coefficient of determination R^2 measures the proportion of the variance of the stock returns explained by the regression model.
For example, if $R^2 = 0.60$, this means that 60% of the observed variance of the stock returns is explained by the linear relationship with the S&P 500 return in the sample considered. The remaining 40% corresponds to the part not explained by this regression.

5. Comparing the three stocks A summary table can be constructed :

```
summary = []
for asset in assets:
model = results[asset]
summary.append({
    'Asset': asset,
    'Alpha': model.params['const'],
    'Beta': model.params['^GSPC'],
    'R2': model.rsquared
})
summary_df = pd.DataFrame(summary)
print(summary_df)
```

The table makes it possible to directly compare the three stocks.
The stock with the highest β is the one whose return is, in the sample studied, the most sensitive to market movements.
Thus, to identify the stock with the greatest exposure to systematic risk, we compare the estimated values of :

$$\beta_{\text{AAPL}}, \qquad \beta_{\text{MSFT}}, \qquad \beta_{\text{JPM}}.$$

However, a high beta should not be confused with a necessarily higher return : beta measures sensitivity to the market factor, not future performance.

6. Graphical representation The relationship between the returns of a stock and those of the market can be represented graphically.

For example, for AAPL :

```
asset = 'AAPL'
Y = returns[asset]
X = returns['^GSPC']
model = results[asset]
plt.figure(figsize=(8, 5))
plt.scatter(X, Y, alpha=0.4)
x_line = np.linspace(X.min(), X.max(), 100)
y_line = model.params['const'] + model.params['^GSPC'] * x_line
plt.plot(x_line, y_line)
plt.xlabel('S&P 500 return')
plt.ylabel('AAPL return')
plt.title('AAPL and market return')
plt.grid(alpha=0.3)
plt.show()
```

Each point represents a daily observation :

$$(R_{M,t}, R_{i,t})\,.$$

The line represents the estimated relationship :

$$\widehat{R}_{i,t} = \widehat{\alpha}_i + \widehat{\beta}_i R_{M,t}.$$

The slope of this line corresponds to the estimated beta $\widehat{\beta}_i$ and its intercept corresponds to the estimated alpha $\widehat{\alpha}_i$.

```
# ================================================================
# EXERCISE 5--CAPM ESTIMATION AND INTERPRETATION
# ================================================================

import numpy as np
import pandas as pd
import yfinance as yf
import statsmodels.api as sm
import matplotlib.pyplot as plt


# ================================================================
# QUESTION 1-- DOWNLOAD THE DATA
# ================================================================

tickers = ['AAPL', 'MSFT', 'JPM', '^GSPC']

data = yf.download(
    tickers,
    start='2021-01-01',
    end='2026-01-01',
    auto_adjust=False
)

# Retrieve adjusted prices
data = data['Adj Close']

print("\n==============================")
print("ADJUSTED PRICES")
print("==============================")
print(data.head())

print("\nMissing values:")
print(data.isna().sum())


# ================================================================
# QUESTION 2--CALCULATE DAILY RETURNS
# ================================================================

returns = data.pct_change().dropna()

print("\n==============================")
print("DAILY RETURNS")
print("==============================")
print(returns.head())


# ================================================================
# QUESTION 3 -- ESTIMATE THE CAPM MODELS
# ================================================================

# List of stocks studied
assets = ['AAPL', 'MSFT', 'JPM']

# Dictionary to store the regression results
results = {}

for asset in assets:

    # Dependent variable:
```

```
    # stock return
    Y = returns[asset]

    # Explanatory variable:
    # market return, approximated here by the S&P 500
    X = returns['^GSPC']

    # Add a constant to estimate alpha
    X_with_const = sm.add_constant(X)

    # OLS regression
    model = sm.OLS(Y, X_with_const).fit()

    # Store the model
    results[asset] = model

    # Display the results
    print("\n=======================================")
    print(f"CAPM REGRESSION : {asset}")
    print("=======================================")

    print(model.summary())


# ================================================================
# QUESTION 4 -- INTERPRET ALPHA, BETA AND R^{2}
# ================================================================

summary = []

for asset in assets:

    model = results[asset]

    # Daily alpha
    alpha_daily = model.params['const']

    # Beta
    beta = model.params['^GSPC']

    # R^2
    r_squared = model.rsquared

    # Approximate annualization of alpha
    alpha_annual = alpha_daily * 252

    summary.append({
        'Asset': asset,
        'Daily alpha': alpha_daily,
        'Annualized alpha': alpha_annual,
        'Beta': beta,
        'R2': r_squared
    })


# Create the summary table
summary_df = pd.DataFrame(summary)

print("\n=======================================")
print("CAPM RESULTS")
print("=======================================")
```

```
123
124 print(summary_df.to_string(index=False))
125
126 # More readable display of the main coefficients
127
128 for asset in assets:
129
130     model = results[asset]
131
132     alpha = model.params['const']
133     beta = model.params['^GSPC']
134     r_squared = model.rsquared
135
136     print(f"\n{asset}")
137     print(f"Daily alpha       : {alpha:.6f}")
138     print(f"Annualized alpha  : {alpha * 252:.2\%}")
139     print(f"Beta              : {beta:.4f}")
140     print(f"R-squared         : {r_squared:.2\%}")
141
142
143 # ================================================================
144 # QUESTION 5 -- COMPARE THE THREE STOCKS
145 # ================================================================
146
147 # Identify the stock with the highest beta
148
149 asset_max_beta = summary_df.loc[
150     summary_df['Beta'].idxmax(),
151     'Asset'
152 ]
153
154 max_beta = summary_df['Beta'].max()
155
156 print("\n========================================")
157 print("BETA COMPARISON")
158 print("========================================")
159
160 print(
161     f"The stock with the highest beta is "
162     f"{asset_max_beta} with a beta of {max_beta:.4f}."
163 )
164
165 # Rank the stocks according to their beta
166
167 beta_comparison = summary_df[
168     ['Asset', 'Beta']
169 ].sort_values(
170     by='Beta',
171     ascending=False
172 )
173
174 print("\nBetas ranked from highest to lowest:")
175 print(beta_comparison.to_string(index=False))
176
177
178 # ================================================================
179 # QUESTION 6 -- GRAPHICAL REPRESENTATION
180 # ================================================================
181
182 for asset in assets:
183
```

```
    # Market return
    X = returns['^GSPC']

    # Stock return
    Y = returns[asset]

    # Model corresponding to the stock
    model = results[asset]

    # X values used to plot the regression line
    x_line = np.linspace(
        X.min(),
        X.max(),
        100
    )

    # Regression line:
    #
    # Y_hat = alpha + beta * X

    y_line = (
        model.params['const']
        + model.params['^GSPC'] * x_line
    )

    # Create the graph
    plt.figure(figsize=(8, 5))

    # Scatter plot
    plt.scatter(
        X,
        Y,
        alpha=0.4
    )

    # Regression line
    plt.plot(
        x_line,
        y_line,
        linewidth=2,
        label='Regression line'
    )

    # Axes
    plt.xlabel('S&P 500 return')
    plt.ylabel(f'{asset} return')

    # Title
    plt.title(
        f'{asset} and market return'
    )

    plt.legend()
    plt.grid(alpha=0.3)

    plt.show()
```

Exercise 4 :

1. Retrieving Fama-French data For this exercise, the factors $Mkt-Rf$, SMB, HML and RF are available in the Kenneth R. French database. You can directly download the monthly data with Python using the following code :

```
import pandas as pd

url = "https://mba.tuck.dartmouth.edu/pages/faculty/\
ken.french/ftp/F-F_Research_Data_Factors_CSV.zip"

ff = pd.read_csv(url, skiprows=3)
```

The variable `ff` then contains the Fama-French data. The main columns of interest are :

```
Mkt-RF    SMB    HML    RF
```

Warning : the values provided by Kenneth French are expressed as percentages. For example : $2.50\% = 0.025$. The factors must therefore be converted to decimal values before being used in the regression :

```
ff[['Mkt-RF', 'SMB', 'HML', 'RF']] /= 100
```

You can then display the first observations with :

```
print(ff.head())
```

2. Retrieving AAPL stock data Then use `yfinance` to retrieve AAPL stock prices over the period January 2021–December 2025.
Using the adjusted prices, calculate the monthly returns of AAPL.

3. Preparing the data You must now bring the two data sources to the same monthly frequency and align them on the same dates :

$$\text{AAPL Price} \longrightarrow \text{AAPL Returns}$$

“‘latex then :

$$\text{AAPL Returns} + \text{Fama-French Factors.}$$

Check that there are no missing values before performing the regressions.

4. CAPM estimation Estimate the model :

$$R_{AAPL,t} - R_{f,t} = \alpha + \beta_{MKT}(Mkt - Rf)_t + \varepsilon_t.$$

Identify α, β_{MKT}, and R^2. Interpret the results.

5. Estimation of the Fama-French model Then estimate :

$$R_{AAPL,t} - R_{f,t} = \alpha + \beta_{MKT}(Mkt - Rf)_t + \beta_{SMB}SMB_t + \beta_{HML}HML_t + \varepsilon_t.$$

Identify the coefficients α, β_{MKT}, β_{SMB} and β_{HML}, as well as R^2.
Interpret the sign and significance of the coefficients β_{SMB} and β_{HML}.

6. Comparison of the two models Construct a comparative table of the results obtained. Answer the following questions :

1. Which model has the higher R^2 ?
2. What does this difference in R^2 mean ?
3. Are the SMB and HML factors statistically significant ?
4. What do the coefficients β_{SMB} and β_{HML} tell us about the profile of the AAPL stock ?
5. Is alpha statistically different from zero ?

```

# ============================================================
# EXERCISE 4 - CAPM AND FAMA-FRENCH ON AAPL
# ============================================================

import pandas as pd
import numpy as np
import yfinance as yf
import statsmodels.api as sm


# ============================================================
# 1. RETRIEVING FAMA-FRENCH DATA
# ============================================================

url = (
    "https://mba.tuck.dartmouth.edu/pages/faculty/"
    "ken.french/ftp/F-F_Research_Data_Factors_CSV.zip"
)

ff = pd.read_csv(url, skiprows=3)

# Rename the first column
ff = ff.rename(columns={ff.columns[0]: "Date"})

# Keep only numerical observations
ff = ff[
    pd.to_numeric(ff["Date"], errors="coerce").notna()
].copy()

# Convert Date to integer
ff["Date"] = ff["Date"].astype(int)

# Keep only the 2021-2025 period
ff = ff[
    (ff["Date"] >= 202101) &
    (ff["Date"] <= 202512)
].copy()

# Convert YYYYMM -> date
ff["Date"] = pd.to_datetime(
    ff["Date"].astype(str),
    format="%Y%m"
)

# Convert percentages to decimals
ff[["Mkt-RF", "SMB", "HML", "RF"]] /= 100

# Set dates to the end of the month
ff["Date"] = ff["Date"] + pd.offsets.MonthEnd(0)
```

```
51
52 # Set Date as the index
53 ff = ff.set_index("Date")
54
55 print("\n=== FAMA-FRENCH FACTORS ===")
56 print(ff.head())
57
58
59 # ============================================================
60 # 2. RETRIEVING AAPL DATA
61 # ============================================================
62
63 aapl = yf.download(
64     "AAPL",
65     start="2021-01-01",
66     end="2026-01-01",
67     auto_adjust=False
68 )
69
70 # Retrieve adjusted price
71 prices = aapl["Adj Close"]
72
73 # End-of-month prices
74 aapl_monthly = prices.resample("ME").last()
75
76 # Monthly returns
77 aapl_returns = aapl_monthly.pct_change()
78
79 # Convert to DataFrame
80 aapl_returns = aapl_returns.to_frame(name="AAPL")
81
82 # Harmonize dates
83 aapl_returns.index = (
84     aapl_returns.index
85     .to_period("M")
86     .to_timestamp("M")
87 )
88
89 print("\n=== AAPL MONTHLY RETURNS ===")
90 print(aapl_returns.head())
91
92
93 # ============================================================
94 # 3. MERGING THE DATA
95 # ============================================================
96
97 data = aapl_returns.join(
98     ff,
99     how="inner"
100 )
101
102 # Remove any missing values
103 data = data.dropna()
104
105 print("\n=== FINAL DATA ===")
106 print(data.head())
107
108 print("\nNumber of observations:", len(data))
109
110 print("\nMissing values:")
111 print(data.isna().sum())
```

```
112
113
114 # ============================================================
115 # 4. AAPL EXCESS RETURN
116 # ============================================================
117
118 data["AAPL_excess"] = data["AAPL"] - data["RF"]
119
120 print("\n=== EXCESS RETURN ===")
121 print(data[["AAPL", "RF", "AAPL_excess"]].head())
122
123
124 # ============================================================
125 # 5. CAPM ESTIMATION
126 # ============================================================
127
128 # Dependent variable
129 y = data["AAPL_excess"]
130
131 # Explanatory variable
132 X_capm = data[["Mkt-RF"]]
133
134 # Add the constant
135 X_capm = sm.add_constant(X_capm)
136
137 # Regression
138 capm = sm.OLS(y, X_capm).fit()
139
140 print("\n")
141 print("=" * 70)
142 print("CAPM RESULTS")
143 print("=" * 70)
144
145 print(capm.summary())
146
147
148 # Retrieve results
149 alpha_capm = capm.params["const"]
150 beta_mkt_capm = capm.params["Mkt-RF"]
151 r2_capm = capm.rsquared
152
153 p_alpha_capm = capm.pvalues["const"]
154
155
156 # ============================================================
157 # 6. ESTIMATION OF THE FAMA-FRENCH MODEL
158 # ============================================================
159
160 # Explanatory variables
161 X_ff = data[[
162     "Mkt-RF",
163     "SMB",
164     "HML"
165 ]]
166
167 # Add the constant
168 X_ff = sm.add_constant(X_ff)
169
170 # Regression
171 ff_model = sm.OLS(y, X_ff).fit()
172
```

```
print("\n")
print("=" * 70)
print("FAMA-FRENCH MODEL RESULTS")
print("=" * 70)

print(ff_model.summary())


# Retrieve results
alpha_ff = ff_model.params["const"]
beta_mkt_ff = ff_model.params["Mkt-RF"]
beta_smb_ff = ff_model.params["SMB"]
beta_hml_ff = ff_model.params["HML"]

r2_ff = ff_model.rsquared

p_alpha_ff = ff_model.pvalues["const"]
p_beta_smb_ff = ff_model.pvalues["SMB"]
p_beta_hml_ff = ff_model.pvalues["HML"]


# ============================================================
# 7. COMPARATIVE TABLE
# ============================================================

results = pd.DataFrame({
    "CAPM": [
        alpha_capm,
        beta_mkt_capm,
        np.nan,
        np.nan,
        r2_capm
    ],

    "Fama-French": [
        alpha_ff,
        beta_mkt_ff,
        beta_smb_ff,
        beta_hml_ff,
        r2_ff
    ]

}, index=[
    "Alpha",
    "Beta MKT",
    "Beta SMB",
    "Beta HML",
    "R^2"
])

print("\n")
print("=" * 70)
print("COMPARATIVE TABLE")
print("=" * 70)

print(results.round(4))


# ============================================================
# 8. P-VALUES TABLE
# ============================================================
```

```
234
235 pvalues = pd.DataFrame({
236     "CAPM": [
237         p_alpha_capm,
238         capm.pvalues["Mkt-RF"],
239         np.nan,
240         np.nan
241     ],
242
243     "Fama-French": [
244         p_alpha_ff,
245         ff_model.pvalues["Mkt-RF"],
246         p_beta_smb_ff,
247         p_beta_hml_ff
248     ]
249
250 }, index=[
251     "Alpha",
252     "Beta MKT",
253     "Beta SMB",
254     "Beta HML"
255 ])
256
257 print("\n")
258 print("=" * 70)
259 print("P-VALUES")
260 print("=" * 70)
261
262 print(pvalues.round(4))
263
264
265 # ============================================================
266 # 9. SIGNIFICANCE TEST
267 # ============================================================
268
269 print("\n")
270 print("=" * 70)
271 print("SIGNIFICANCE TESTS")
272 print("=" * 70)
273
274 # Alpha
275 if p_alpha_ff < 0.05:
276     print("Alpha: significant at the 5% level.")
277 else:
278     print("Alpha: not significant at the 5% level.")
279
280 # SMB
281 if p_beta_smb_ff < 0.05:
282     print("SMB: significant at the 5% level.")
283 else:
284     print("SMB: not significant at the 5% level.")
285
286 # HML
287 if p_beta_hml_ff < 0.05:
288     print("HML: significant at the 5% level.")
289 else:
290     print("HML: not significant at the 5% level.")
291
292
293 # ============================================================
294 # 10. COMPARISON OF R^2
```

```
# ============================================================

print("\n")
print("=" * 70)
print("COMPARISON OF R^2")
print("=" * 70)

print("R^2 CAPM         :", round(r2_capm, 4))
print("R^2 Fama-French  :", round(r2_ff, 4))
print("Difference       :", round(r2_ff - r2_capm, 4))
```

Exercise 5 — PCA and Statistical Factor Model

Consider a portfolio composed of the following eight stocks :

$$\texttt{AAPL},\ \texttt{MSFT},\ \texttt{AMZN},\ \texttt{NVDA},\ \texttt{JPM},\ \texttt{V},\ \texttt{XOM},\ \texttt{PG}.$$

The objective is to identify the main statistical factors common to the returns of these stocks.

1. Download the adjusted prices of the eight stocks over the period from January 1, 2021 to January 1, 2026.
2. Calculate the daily return matrix :

$$R_t = \begin{pmatrix} R_{AAPL,t} \\ R_{MSFT,t} \\ \vdots \\ R_{PG,t} \end{pmatrix}.$$

3. Apply PCA to the returns and retain the first three principal components.
4. For each component, calculate the proportion of explained variance :

$$\frac{\lambda_k}{\sum_{j=1}^{N} \lambda_j}.$$

5. Plot the variance explained by PC_1, PC_2 and PC_3.
6. Extract the factor loadings (*loadings*) of PC_1 and plot them.
7. Examine the signs and magnitudes of the loadings of PC_1. Can this component be interpreted as a common movement of the stocks ?
8. Then construct the time series of the three factors :

$$PC_{1,t}, \qquad PC_{2,t}, \qquad PC_{3,t}.$$

9. For the stock `AAPL`, estimate the factor model :

$$R_{AAPL,t} = \alpha + \beta_1 PC_{1,t} + \beta_2 PC_{2,t} + \beta_3 PC_{3,t} + \varepsilon_t.$$

10. Interpret the coefficients β_1, β_2 and β_3.